

Tartalom

Bevezetés.....	1
1.1 A téma aktualitása és jelentősége	1
1.2 A kutatás problémamegfogalmazása	1
1.3 Kutatási rés azonosítása	2
1.4 Kutatási célok és hipotézisek	2
1.5 Kutatási módszertan	3
1.6 A kutatás korlátai	4
1.7 A dolgozat felépítése	5
1.8 Tudományos és gyakorlati hozzájárulás.....	6
2. Az IoT technológiai alapjai	6
2.1. Az IoT fogalma és jelentősége.....	6
2.2. IoT architektúrák és modelljei.....	7
2.3. Kommunikációs technológiák és protokollok.....	9
2.4. Biztonsági protokollok alkalmazásának szakmai indoklása	10
3. Az IoT biztonság sajátosságai	11
3.1. Összegzés	12
3.2. Támadási felületek kategorizálása IoT környezetben	12
3.3. Összegzés	15
4. Biztonságos protokollok és védelmi mechanizmusok	15
4.1 A TLS 1.3 szerepe és alkalmazása IoT környezetben	15
4.1.1 A TLS 1.3 protokoll főbb sajátosságai	15
4.1.2 TLS 1.3 IoT-eszközökön: előnyök és korlátok.....	16
4.1.3 TLS 1.3 és Zero Trust elvek kapcsolata	17
4.2 Biztonságos MQTT kommunikáció TLS 1.3 felett.....	17
4.2.1 Az MQTT biztonsági modellje és támadási felületei	17
4.2.2 MQTT over TLS 1.3: csatornaszintű védelem.....	18
4.2.3 Hozzáférés-szabályozás és topic-szintű védelem.....	19
4.2.4 Kapcsolódás a dolgozat gyakorlati részéhez	19
4.3 Zero Trust alapelvek és alkalmazásuk IoT környezetben	20
4.3.1 A Zero Trust modell fogalma és alapelvei.....	20
4.3.2 Zero Trust IoT-rendszerekben: identitás, eszközállapot és hálózati szegmentáció .	21
4.3.3 Zero Trust és defense-in-depth IoT környezetben.....	22
4.3.4 Kapcsolódás a dolgozat kutatási kérdéseire és gyakorlati részéhez	22
4.4 A vizsgált biztonsági megoldások és a szimulációs kutatás kapcsolata	23
4.4.1 Kapcsolódás a kutatási kérdésekhez és hipotézisekhez	23
4.4.2 A biztonsági megoldások leképezése a szimulációs környezetre.....	24

4.4.3 Korlátok és további bővítési lehetőségek	25
5. A szimulációs környezet és mérési módszertan	26
5.1 A szimulációs környezet felépítése	26
5.1.1 A környezet fő komponensei	26
5.1.2 Hardver- és szoftverkörnyezet.....	27
5.1.3 Hálózati topológia és MQTT-topic struktúra	28
5.1.4 A mérési folyamat lépései	29
5.2 MQTT-alapú mérési backend megvalósítása	30
5.2.1 Követelmények és szerepkör.....	31
5.2.2 Titkosítatlan collector modul (collector.py)	31
5.2.3 TLS-sel védett collector modul (collector_tls.py).....	33
5.2.4 Tesztpublikáló szkriptek (pub_test.py, pub_test_tls.py).....	34
5.2.5 Elemző modul (analyze_measurements.py).....	34
5.3 Mérési módszertan és vizsgált scenáriók	35
5.3.1 Vizsgált brokerkonfigurációk	35
5.3.2 Mérőszámok és kiértékelési szempontok	36
5.3.3 Mérési eljárás	37
5.3.4 Támadási scenáriók	38
5.4 Mérési eredmények és értékelés.....	39
5.4.1 Baseline – titkosítatlan MQTT (Konfiguráció A)	39
5.4.2 TLS 1.3 feletti MQTT szerverhitelesítés (Konfiguráció B)	40
5.4.3 Összehasonlító értékelés.....	42
6. A fejlesztett (szimulált) IoT-biztonsági rendszer bemutatása	43
6.1 Architektúra logikai felépítése	43
6.2 ESP32-alapú szenzor node-ok implementációja	46
6.2.1 WiFi-kapcsolat inicializálása Wokwi környezetben.....	46
6.2.2 MQTT-kliens inicializálása és kapcsolatfelépítés	47
6.2.3 JSON payload és topic-struktúra.....	48
6.2.4 Időzítés és adatszórás: 5 másodperces periódus.....	49
6.2.5 sensor1 és sensor2 szerepe a mérésekben	50
6.3 MQTT-broker és Mosquitto-konfigurációk.....	51
6.3.1 Konfiguráció A – titkosítatlan MQTT (1883)	51
6.3.2 Konfiguráció B – TLS 1.3 feletti MQTT (8883).....	52
6.3.3 Konfiguráció C – mTLS + ACL (konceptcionális bővítés).....	53
7. Támadási scenáriók, tesztelés és mérési eredmények – II.....	54
7.1 Normál működés biztonságos konfigurációban	55
7.1.1 Szenárió és környezet rövid áttekintése	55

7.1.2 Üzenetküldési ráta és időbeli viselkedés	56
7.1.3 Hőmérsékleti értékek eloszlása és konzisztenciája	57
7.1.4 Biztonsági megfigyelések és következtetések.....	58
7.2 Jogosulatlan publish titkosítatlan MQTT-n (Konfiguráció A)	59
7.2.1 Támadási modell és cél	59
7.2.2 A támadó kliens implementációja (pub_test.py)	60
7.2.3 Hatás a collector és analyzer nézőpontjából.....	61
7.2.4 Biztonsági következtetések titkosítatlan MQTT mellett	62
7.3 Jogosulatlan publish TLS felett (Konfiguráció B)	62
7.3.1 Támadási modell TLS-es környezetben	63
7.3.2 A támadó kliens TLS-es változata	63
7.3.3 A támadás hatása a TLS-es mérési adatokra	64
7.3.4 Biztonsági tanulságok TLS mellett	65
7.4 mTLS + ACL alapú védelem várható hatása (Konfiguráció C – koncepcionális elemzés)	66
7.4.1 Védelmi modell: kliensidentitás és minimális jogosultság	66
7.4.2 A jogosulatlan publish várható kimenetele mTLS + ACL mellett	67
7.4.3 Implementációs szempontok és korlátok	68
7.5 MI-alapú anomáliadetektálási kísérlet	70
7.6 Összefoglaló értékelés és konfigurációk összehasonlítása.....	71
7.6.1 Konfiguráció A: „nyitott” MQTT-busz	72
7.6.2 Konfiguráció B: TLS 1.3 – csatornavédelem, de korlátozott védelem a támadók ellen	72
7.6.3 Konfiguráció C: mTLS + ACL – a támadási felület jelentős szűkítése	73
7.6.4 Kapcsolat a kutatási kérdésekkel és további irányok	73
8. Összegzés és kitekintés	74
8.1 Eredmények összefoglalása.....	74
8.2 A kutatási kérdések és hipotézisek kiértékelése	76
8.3 További kutatási irányok	77
9. Felhasznált irodalomjegyzék.....	78

Bevezetés

1.1 A téma aktualitása és jelentősége

Az **Internet of Things (IoT)** napjaink egyik legmeghatározóbb technológiai trendje, amely forradalmasítja az ipart, a háztartásokat és a mindennapi élet számos területét. Az IoT-eszközök elterjedése rohamos ütemben zajlik: a Statista 2024-es előrejelzése szerint a globális IoT-eszközök száma 2030-ra elérheti a **75 milliárdot**, szemben a 2020-as **31 milliárddal** (Statista, 2024, p. 12). Ez a növekedés minden életterületet áthat: az okosotthon-megoldásoktól kezdve az ipari automatizáláson és egészségügyi monitorozó rendszereken át a közlekedési és energetikai hálózatokig az IoT mára átszővi a társadalom infrastruktúráját. (Statista, 2024).

Magyarország esetében az IoT penetráció 2023-ban elérte a 8,2 millió aktív eszközt, és az előrejelzések szerint 2028-ra ez a szám meghaladja a 15 milliót (NMHH, 2023, p. 45). Ez azt jelenti, hogy átlagosan minden magyar háztartásban 2-3 IoT-eszköz található, ami jelentős biztonsági kockázatokat hordoz.

1.2 A kutatás problémamegfogalmazása

Ezzel a gyors fejlődéssel azonban új, korábban ismeretlen **biztonsági kihívások** jelentek meg. Az Európai Hálózat- és Információbiztonsági Ügynökség (ENISA) 2023-as jelentése szerint az IoT-kapcsolatos biztonsági incidensek száma **2019 óta évente átlagosan 40%-kal növekszik**. Az ismert kibertámadások, mint a **2016-os Mirai botnet – amely több mint 600 000 IoT-eszközt kompromittált és 1,2 Tbps sebességű DDoS támadást generált** (Antonakakis et al., 2017) vagy a **2021-es Realtek SDK sebezhetőség – amely több millió eszközt tett támadhatóvá 65 különböző gyártónál (CVE-2021-35394), 2023-as TP-Link router támadási hullám – ahol több mint 2 millió eszköz került kompromittálásra Európában** (CERT-EU, 2023), világosan mutatják, hogy az IoT biztonsági kérdései nemcsak technológiai, hanem **gazdasági, társadalmi és nemzetbiztonsági dimenzióval** is bírnak (ENISA, 2023).

Az IoT rendszerek sajátosságai – az **erőforrás-korlátok**, a **heterogén környezet**, a **gyors fejlesztési ciklusok** és a **fizikai hozzáférhetőség** – olyan egyedi **támadási felületeket** teremtenek, amelyek túlmutatnak a hagyományos IT-biztonság keretein. Erre reagálva az **ETSI EN 303 645** szabvány 2020-ban világelsőként rögzítette azokat a **minimális biztonsági követelményeket**, amelyeket a fogyasztói IoT-eszközöknek teljesíteniük kell (például alapértelmezett jelszavak tiltása, rendszeres szoftverfrissítés, sebezhetőségi bejelentési

folyamat) (ETSI, 2020). Mindez azonban csak részben képes kezelni az IoT biztonsági problémáinak komplexitását.

A dolgozat központi problémája tehát az, hogy **milyen sebezhetőségek jellemzik leginkább az IoT-eszközöket, és ezek ellen milyen hálózati szintű védelmi mechanizmusok és architektúrák képesek hatékony megoldást nyújtani.**

1.3 Kutatási rés azonosítása

A jelenlegi szakirodalom három fő területen mutat hiányosságokat:

1. **Gyakorlati összehasonlító elemzések hiánya:** Bár számos elméleti munka foglalkozik az IoT biztonsági protokollokkal, kevés az olyan kutatás, amely valós környezetben méri össze a TLS 1.3, DTLS 1.2/1.3 és OSCORE teljesítményét erőforrás-korlátozott eszközökön (Rescorla, 2018; Raza et al., 2017).
2. **Integrált védelmi keretrendszerek:** A legtöbb kutatás egy-egy konkrét támadási vektorra fókuszál, de hiányzik az átfogó, többretegű védelmi architektúra (NISTIR 8259, 2020).
3. **Magyar nyelvű szakirodalom:** A hazai IoT biztonsági kutatások még gyerekcipőben járnak, különösen a hálózati protokollok területén.

1.4 Kutatási célok és hipotézisek

Fő kutatási kérdés:

Milyen sebezhetőségek jellemzik leginkább az IoT-eszközöket, és ezek ellen milyen hálózati szintű védelmi mechanizmusok és architektúrák képesek hatékony megoldást nyújtani?

Alkérdések:

1. Mely kriptográfiai protokollok (TLS 1.3, DTLS 1.2/1.3, OSCORE) nyújtják a legjobb kompromisszumot a biztonság és teljesítmény között?
2. Hogyan implementálható a Zero Trust modell IoT környezetben?
3. Milyen automatizált védelmi mechanizmusok növelhetik a fenyegetések felismerésének hatékonyságát?

Hipotézisek:

- **H1:** Az IoT rendszerek támadási felületei elsősorban a nem megfelelően implementált hitelesítési mechanizmusokból, a gyenge alapértelmezett konfigurációkból és az elavult kriptográfiai módszerek használatából erednek.
- **H2:** A modern biztonságos protokollok (TLS 1.3, DTLS 1.2/1.3, CoAP security módok) megfelelő optimalizációkkal való adaptálása jelentősen csökkentheti az IoT-eszközök hálózati szintű sebezhetőségét.
- **H3:** A Zero Trust biztonsági modell IoT-ra adaptált változata és a többrétegű védelmi mechanizmusok (defense-in-depth) kombinációja hatékony védelmet nyújt a fejlett perzisztens fenyegetések ellen.
- **H4:** Az automatizált sebezhetőség-felismerés és az MI-alapú anomáliadetekció integrációja javítja a biztonsági incidensek korai felismerésének hatékonyságát.

1.5 Kutatási módszertan

A kutatás módszertana két egymásra épülő részre tagolódik: egy szisztematikus szakirodalmi feldolgozásra, valamint egy kontrollált, laboratóriumi jellegű szimulációs környezetben végzett gyakorlati vizsgálatra. A dolgozat célja nem csupán az IoT-biztonság elméleti hátterének áttekintése, hanem annak bemutatása is, hogy az egyes védelmi mechanizmusok egy konkrét, MQTT-alapú környezetben milyen módon befolyásolják a rendszer viselkedését és a támadási felület nagyságát.

A szakirodalmi rész Systematic Literature Review szemléletben készült. A feldolgozott források között tudományos publikációk, szabványok, iparági ajánlások és nemzetközi irányelvek szerepelnek. A keresés az IoT security, MQTT security, TLS 1.3, DTLS, OSCORE, Zero Trust és anomaly detection kulcsszavak köré szerveződött. A szakirodalmi feldolgozás célja az volt, hogy rendszerezett módon feltárja az IoT-rendszerek leggyakoribb sebezhetőségeit, a tipikus támadási mintákat, valamint azokat a védelmi mechanizmusokat, amelyek a hálózati és alkalmazási rétegben reálisan alkalmazhatók erőforrás-korlátozott környezetben is.

A gyakorlati vizsgálatok egy reprodukálható, laboratóriumi jellegű szimulációs környezetben zajlottak. A szenzorréteget két darab, Wokwi környezetben futó ESP32-alapú node reprezentálta, amelyek MQTT-kliensként működve periodikusan hőmérsékleti adatokat

publikáltak. A kommunikáció központi eleme egy Eclipse Mosquitto broker volt, amely több konfigurációban került vizsgálatra: titkosítatlan MQTT-kapcsolattal, TLS 1.3-mal védett környezetben, valamint koncepcionális szinten mTLS és ACL alapú kiterjesztéssel. Az adatgyűjtést Pythonban írt collector modulok, az elemzést pedig külön analyze szkriptek támogatták.

A mérések során rögzített főbb mutatók az üzenetszám, a futási időtartam, az üzenetküldési ráta, valamint az átlagos hőmérsékleti értékek voltak. Ezek lehetővé tették a baseline és a TLS-es konfigurációk összehasonlítását, továbbá annak vizsgálatát, hogy a kommunikációs csatorna védelme milyen mértékű többletterheléssel jár a szimulált IoT-rendszerben. A gyakorlati rész kiterjedt egyszerű támadási scenáriók vizsgálatára is, különösen a jogosulatlan publish esetre, amikor egy külső kliens legitim szenzor topicjára próbál hamis adatokat publikálni.

A dolgozat kiegészítő kutatási elemeként egy egyszerű, gépi tanulás alapú anomáliadetektálási kísérlet is készült. Ennek során egy Isolation Forest modell került kialakításra, amely a normál és támadó MQTT-forgalomból előállított jellemzők – a szenzorazonosító, a hőmérsékleti érték és az egymást követő üzenetek közötti időeltérés – alapján jelölte az anomáliagyánús mintákat. A modell kiértékelése kontrollált, címkézett adathalmazon történt, és a pontosság, precision, recall, valamint F1-score mutatók alapján került értékelésre.

A módszertan logikája ennek megfelelően a következő: a szakirodalom alapján azonosításra kerültek az IoT-rendszerek legfontosabb támadási felületei és védelmi megoldásai, majd ezek közül a dolgozat fókuszának megfelelő elemek gyakorlati, mérhető formában is vizsgálatra kerültek. Ez a megközelítés biztosítja, hogy a dolgozat ne pusztán leíró jelleggel tárgyalja az IoT-biztonság kérdéseit, hanem saját mérésekkel és elemzésekkel is alátámassza a megfogalmazott következtetéseket.

1.6 A kutatás korlátai

Technikai korlátok:

- A mérések laboratóriumi környezetben történnek, ami nem tükrözi teljesen a valós működési feltételeket
- Korlátozott eszközparkból adódó reprezentativitási kérdések
- Egyéni kutatóként időbeli korlátok

Módszertani korlátok:

- Nem kerül sor nagy volumenű (10,000+ eszköz) skálázhatósági tesztekre
- A biztonsági tesztek etikai okokból kizárólag saját eszközökön történnek

1.7 A dolgozat felépítése

A dolgozat a bevezető fejezetet követően hét további fő fejezetből áll.

A 2. fejezet az IoT technológiai alapjait mutatja be. Ebben a részben kerül sor az IoT fogalmi keretrendszerének, architektúráinak, kommunikációs modelljeinek és a dolgozat szempontjából releváns hálózati protokollok technológiai hátterének áttekintésére. Ez a fejezet teremti meg a későbbi biztonsági elemzések műszaki alapját.

A 3. fejezet az IoT-biztonság fő kihívásait és támadási felületeit tárgyalja. Bemutatja az IoT-rendszerekre jellemző sebezhetőségeket, a tipikus támadási vektorokat, valamint több olyan ismert esettanulmányt és incidensmintát, amelyek jól szemléltetik a témakör gyakorlati jelentőségét.

A 4. fejezet a biztonságos protokollokat és védelmi mechanizmusokat rendszerezi. Itt hangsúlyt az MQTT és a CoAP biztonsági rétegeinek elemzése, különösen a TLS 1.3, a DTLS és az OSCORE vizsgálata, valamint a Zero Trust és defense-in-depth szemlélet IoT-környezetre történő értelmezése.

Az 5. fejezet a szimulációs környezetet és a mérési módszertant ismerteti. Bemutatja a Wokwi-ban futó ESP32 node-okat, a Mosquitto broker konfigurációit, a Python-alapú collector és analyzer modulokat, valamint a baseline és TLS-es mérések eredményeit és ezek értékelését.

A 6. fejezet a fejlesztett, szimulált IoT-biztonsági rendszer logikai és gyakorlati felépítését írja le. Részletesen tárgyalja az architektúra komponenseit, az MQTT-kommunikációhoz kapcsolódó konfigurációkat, valamint a mTLS + ACL alapú továbbfejlesztés koncepcionális modelljét.

A 7. fejezet a támadási scénáriók és a második körös empirikus eredmények bemutatására szolgál. Ebben a részben kerül sor a jogosulatlan publish scénáriók elemzésére különböző biztonsági konfigurációk mellett, továbbá itt kap helyet az MI-alapú anomáliadetektálási kísérlet eredményeinek ismertetése is.

A 8. fejezet összegzi a dolgozat fő megállapításait, értékeli a kutatási hipotéziseket, és kijelöli a további kutatási irányokat. Ez a rész foglalja össze, hogy a dolgozat milyen módon járult hozzá az IoT-biztonság gyakorlati és elméleti kérdéseinek jobb megértéséhez.

1.8 Tudományos és gyakorlati hozzájárulás

Tudományos értékek:

- Empirikus adatok erőforrás-korlátozott eszközök teljesítményéről
- Új integrált védelmi keretrendszer javaslata

Gyakorlati értékek:

- Implementációs útmutatók fejlesztők számára
- Biztonsági checklist IoT projekt vezetőknek
- Konkrét konfigurációs javaslatok

2. Az IoT technológiai alapjai

2.1. Az IoT fogalma és jelentőséges

Az IoT definíciói

Az Internet of Things (IoT) fogalma a 2000-es évek elején került be a szakirodalomba, és azóta folyamatosan bővül és változik a technológiai fejlődés ütemének megfelelően. Az IoT koncepciójának lényege, hogy a fizikai eszközök hálózati kapcsolattal és beágyazott intelligenciával rendelkeznek, ezáltal képesek adatokat gyűjteni, továbbítani, feldolgozni, és autonóm döntéseket támogatni. Bár a definíciók eltérő hangsúlyokkal élnek, közös bennük, hogy a fizikai és virtuális világ szoros összekapcsolódását írják le.

ITU (International Telecommunication Union): az IoT-t egy „globális infrastruktúraként” határozza meg, amely az információs társadalom számára biztosítja a fizikai és virtuális dolgok összekapcsolását, interoperábilis ICT-technológiákon keresztül. Ez a megközelítés erősen infrastruktúra-centrikus, a hálózati háttér kiépítésére helyezi a hangsúlyt.

NIST (National Institute of Standards and Technology): az IoT-t olyan hálózati infrastruktúraként írja le, amely a fizikai objektumokat és virtuális entitásokat intelligens interfészekon keresztül kapcsolja össze. Ez a definíció inkább hálózati és interfész-orientált, vagyis a kapcsolat és integráció mikéntjét emeli ki.

ISO/IEC 30141: az IoT-t olyan infrastruktúraként definiálja, amely a fizikai és virtuális dolgok összekapcsolása révén fejlett szolgáltatásokat tesz lehetővé. Ez a megközelítés szolgáltatás- és interoperabilitás-központú, tehát a gyakorlati alkalmazás és a szolgáltatások létrehozása kerül előtérbe.

A definíciók közötti eltérések jól mutatják, hogy az IoT nem egyetlen technológiát jelent, hanem egy sokrétegű ökoszisztémát. Az ITU inkább a globális keretrendszerre fókuszál, a NIST a technológiai integrációra, míg az ISO/IEC a szolgáltatásalapú szemléletet hangsúlyozza.

Az IoT alapvető jellemzői

Az IoT ökoszisztéma három alapvető jellemzővel írható le:

Eszközök hálózatba kapcsolása: fizikai objektumok (pl. érzékelők, aktuátorok, beágyazott rendszerek) internetes és helyi hálózati kapcsolattal való felruházása, amely lehetővé teszi az adatok folyamatos áramlását.

Adatgyűjtés és -feldolgozás: a csatlakoztatott eszközök által generált adatok rögzítése, előfeldolgozása és továbbítása nagyobb rendszerek, például felhőszolgáltatások vagy központi vezérlők felé.

Automatizáció és intelligens döntéshozatal: az összegyűjtött adatok alapján automatikus műveletek indítása, prediktív elemzések végrehajtása és önálló döntéshozatali mechanizmusok működtetése.

E három pillér együttesen teszi lehetővé, hogy az IoT a társadalmi és gazdasági folyamatok szerves részévé váljon, ugyanakkor ezek a jellemzők egyben biztonsági kihívásokat is generálnak – például az adatok bizalmasságának, integritásának és rendelkezésre állásának védelmét.

2.2. IoT architektúrák és modelljei

Az IoT rendszerek összetettsége miatt többféle architektúra-modell született, amelyek eltérő absztrakciós szinteken szemléltetik a technológiai rétegeket és funkciókat.

Háromrétegű architektúra

A **klasszikus háromrétegű modell** az IoT rendszerek legegyszerűbb struktúráját írja le:

Érzékelési réteg (Perception Layer): a fizikai szenzorok és aktuátorok szintje, ahol az adatok keletkeznek. Példák: hőmérséklet- és páratartalom-érzékelők, RFID-címkék, IP-kamerák.

Hálózati réteg (Network Layer): felelős az adatok továbbításáért és routingjáért a különböző protokollok segítségével. Tipikus technológiák: WiFi, ZigBee, 4G/5G mobilhálózatok.

Alkalmazási réteg (Application Layer): itt történik az adatok feldolgozása, megjelenítése, illetve a felhasználók és más rendszerek számára történő szolgáltatásnyújtás.

A háromrétegű modell jól használható alapfogalomként, de a modern IoT rendszerek összetettségét nem írja le teljes körűen.

Ötrétegű kiterjesztett architektúra

A **kiterjesztett ötrétegű modell** pontosabb képet ad az IoT működéséről, különösen az adattárolás és feldolgozás fontosságát hangsúlyozva:

Érzékelési réteg – mint a háromrétegű modellben.

Hálózati réteg – kibővítve edge–cloud kapcsolatokkal.

Perzisztencia réteg (Data Storage Layer): az adatok rövid és hosszú távú tárolását biztosítja (adatbázisok, felhőszolgáltatások, backup rendszerek).

Feldolgozási réteg (Data Processing Layer): az adatok elemzését, feldolgozását és intelligens döntéstámogatását valósítja meg. Ide tartozik a mesterséges intelligencia és gépi tanulás integrációja is.

Alkalmazási réteg – a felhasználói szolgáltatások és API-k biztosítása.

Ez a modell jobban tükrözi az IoT valóságát, különösen a nagy adatmennyiségek és valós idejű feldolgozás szempontjából.

Edge Computing és Fog Computing szerepe

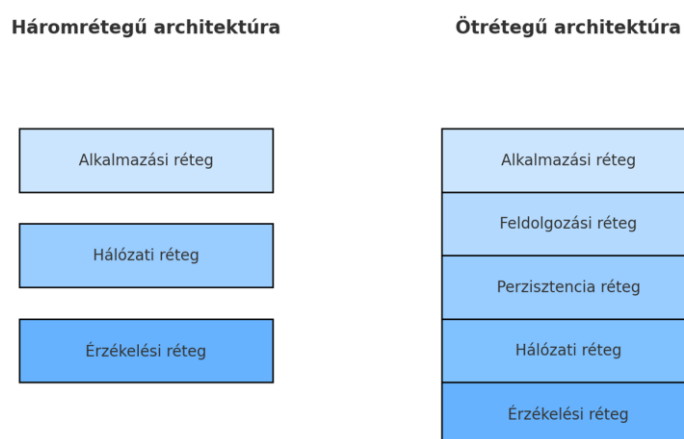
Az utóbbi években a felhőalapú feldolgozás korlátai (nagy latencia, hálózati túlterheltség) miatt előtérbe került az edge computing és a fog computing koncepciója.

Edge Computing: az adatfeldolgozás az IoT-eszközök közelében, helyben történik. Ez csökkenti a hálózati késleltetést, csökkenti a sávszélesség-igényt, és növeli a biztonságot azáltal, hogy az adatok nem minden esetben kerülnek központi szerverre. Példák: okoskamerák helyi képfelismeréssel, ipari robotok valós idejű döntéshozatallal.

Fog Computing: a feldolgozás köztes szinten zajlik, az edge és a cloud között. A fog réteg regionális adatközpontokból és gateway-ekből áll, amelyek elosztott módon végzik az adatok feldolgozását és továbbítását. Ez a modell jobb skálázhatóságot és terhelésmegosztást biztosít, és lehetővé teszi, hogy a kritikus döntések közelebb történjenek az adatforráshoz.

Mindkét megközelítés hozzájárul a megbízhatóság növeléséhez és a támadási felületek szűkítéséhez, hiszen az adatok nem kizárólag a központi hálózatban, hanem részben helyi környezetben kerülnek feldolgozásra.

2.1 ábra - IoT három- és ötrétegű architektúra összehasonlítása



2.3. Kommunikációs technológiák és protokollok

Az IoT kommunikációs technológiák és protokollok sokfélesége jól ismert a szakterület szereplői számára, ezért a jelen fejezet nem a technológiák részletes bemutatását, hanem a dolgozat szempontjából releváns választások indoklását tartalmazza. A kutatás központi kérdése szempontjából a kommunikációs protokollok vizsgálata azért szükséges, mert a támadási felületek túlnyomó része a hálózati kommunikációhoz köthető (Sicari et al., 2015; ENISA, 2023).

Az IoT rendszerek kommunikációs megoldásai három szempontból bírnak jelentőséggel a biztonsági vizsgálatok során:

1. Energia- és erőforrás-korlátok

Az erőforrás-korlátozott eszközök nem képesek nagy kriptográfiai overhead kezelésére, ezért olyan protokollokra van szükség, amelyek könnyű implementációval és optimalizált adatcserével működnek (pl. MQTT, CoAP). A szakirodalom kiemeli, hogy az IoT környezetben különösen fontos a lightweight biztonsági mechanizmusok alkalmazása (Rahman & Yaqoob, 2018).

2. Hálózati modell és architektúra

Az MQTT publish–subscribe modellje és a CoAP request–response működése eltérő támadási felületeket eredményez. Az MQTT brokeralapú architektúrája érzékeny például MITM és session-hijacking típusú támadásokra (Chakrabarty & Watanabe, 2017), míg a CoAP UDP-alapú működését gyakran érik replay vagy spoofing támadások (Shelby et al., 2014).

3. Biztonsági réteg integrálhatósága

A modern IoT biztonsági mechanizmusok például a TLS 1.3, DTLS 1.2/1.3 vagy OSCORE eltérő módon integrálhatók a protokollstruktúrába. A különböző kriptográfiai modellek összehasonlítását több nemzetközi ajánlás is javasolja (NISTIR 8259, 2020; ETSI EN 303 645, 2020).

A továbbiakban ezért az MQTT és a CoAP protokollok kerülnek részletes vizsgálatra, mivel ezek alkalmasak a kutatási hipotézisek tesztelésére, iparági standardok, és széles körben használt technológiák mind fogyasztói, mind ipari környezetben.

2.4. Biztonsági protokollok alkalmazásának szakmai indoklása

A dolgozatban vizsgált TLS 1.3, DTLS 1.2/1.3 és OSCORE protokollok nem önmaguk miatt fontosak, hanem azért, mert különböző módon biztosítanak védelmet az IoT kommunikáció során ezt számos kutatás is alátámasztja (Rescorla, 2018; Raza et al., 2017; Bhardwaj et al., 2020).

TLS 1.3 A TLS 1.3-at széles körben ajánlják IoT környezetekben a csökkentett handshake és modern kriptográfia miatt (Rescorla, 2018).

- optimális TCP-alapú IoT kommunikációhoz
- minimalizált késleltetés
- az MQTT szabványos biztonsági rétege (OASIS MQTT Standard)

DTLS 1.2/1.3

A DTLS az UDP-alapú protokollok, de facto biztonsági megoldása (Rescorla & Modadugu, 2012).

- CoAP biztonságának alapja
- ellenáll a csomagvesztés és reordering okozta problémáknak
- hatékony erőforrás-kezelés

OSCORE

Az OSCORE az objektumszintű titkosítást vezeti be, amely könnyűsúlyú, IoT-ra optimalizált védelem (Raza et al., 2017).

- end-to-end titkosítás több proxy-n keresztül is
- minimális overhead
- CoAP környezetben iparági standard (RFC 8613)

A különböző protokollok vizsgálata módszertani szükségszerűség, mivel eltérő támadási modellek, előnyök és kompromisszumok tesztelhetők velük.

3. Az IoT biztonság sajátosságai

Az IoT rendszerek biztonsági kihívásai alapvetően a decentralizált működésből, az erőforrás-korlátokból és a heterogén ökoszisztémából erednek (Sicari et al., 2015; ENISA, 2023). A klasszikus IT-architektúrákkal szemben az IoT egy sokkal kevésbé kontrollált környezetet jelent, amely több támadási lehetőséget biztosít a fenyegetők számára.

1. Erőforrás-korlátozott eszközök

A legtöbb IoT-eszközben kevés CPU, memória és energiatartalék áll rendelkezésre, ami korlátozza a kriptográfiai képességeket. Ezt a biztonsági problémát széles körben elemzi a szakirodalom (Weber & Studer, 2016).

2. Heterogén ökoszisztéma és interoperabilitási kihívások

A sokféle platform, protokoll és operációs rendszer összehangolása gyakran hibás konfigurációkhoz és kompatibilitási problémákhoz vezet (NISTIR 8259, 2020).

3. Fizikai hozzáférésből fakadó kockázatok

A fizikai támadások például firmware dump, debug interfészek elérése, kulcskinyerés az IoT környezetben sokkal gyakoribbak, mint hagyományos IT-rendszerekben (Costin et al., 2014).

4. Gyors fejlesztési ciklus és rövid termékéletciklus

Az olcsó tömeggyártás és gyors piaci megjelenés gyakran a biztonság rovására megy (Sicari et al., 2015).

Gyakoriak a hardcoded jelszavak vagy a frissítés nélküli firmware-ek.

5. Decentralizált és skálázható infrastruktúrák

A nagy eszközszám könnyen botnetek kiépítéséhez vezethet a Mirai-botnet klasszikus példa erre (Antonakakis et al., 2017).

6. Kritikus adatok valós idejű kezelése

Az egészségügyi vagy ipari IoT-rendszerek esetén kiemelten fontos a késleltetés, integritás és rendelkezésre állás biztosítása (ENISA, 2023).

3.1. Összegzés

Az IoT biztonsági sajátosságai meghatározzák azokat a támadási felületeket, amelyek később részletes elemzésre kerülnek. A heterogén környezet, a fizikai hozzáférhetőség és az erőforráskorlátok együtt egy olyan támadói környezetet teremtenek, amelyben komplex, többrétegű védelmi mechanizmusokra van szükség.

3.2. Támadási felületek kategorizálása IoT környezetben

Az IoT rendszerek sebezhetőségeinek megértéséhez szükséges a támadási felületek pontos kategorizálása. A támadási felület (attack surface) azon összes komponens és interfész együttese, amelyen keresztül egy támadó kölcsönhatásba léphet a rendszerrel. A szakirodalom az IoT támadási felületét tipikusan **négy fő réteg** szerint írja le: eszközszint, hálózati szint, alkalmazási szint, valamint a felhő- és backend infrastruktúra szintje (ENISA, 2023; NISTIR 8259, 2020).

Az alábbi kategorizálás a nemzetközi szabványok és biztonsági ajánlások (pl. OWASP IoT Top 10, ETSI EN 303 645) alapján készült, és a későbbi fejezetekben elemzett támadási vektorok alapját képezi.

1. Eszközszintű támadási felületek

Az eszközszint jelenti az IoT infrastruktúra legalsó rétegét, amely magában foglalja a fizikai hardverkomponenseket, a beágyazott firmware-t és a lokális interfészeket. A támadások leggyakoribb formái:

• Fizikai manipuláció és hozzáférés

- debug interfészek (UART, JTAG) visszafejtése
- firmware dump készítése
- titkosítási kulcsok fizikai kinyerése
- secure boot megkerülése

A fizikai hozzáférésből fakadó fenyegetések IoT-ben különösen relevánsak, mivel sok eszköz felügyelet nélkül üzemel (Sicari et al., 2015).

• Hardcoded jelszavak, gyenge hitelesítés

Az OWASP IoT Top 10 első számú hibája: *Insecure Default Passwords* (OWASP, 2022).

• Firmware sérülékenységek

Gyakoriak a:

- frissítési mechanizmus hiánya,
- aláíratlan firmware-csomagok,
- sérülékeny könyvtárverziók.

A 2021-es Realtek SDK sebezhetőség több mint 60 gyártó eszközét érintette (ENISA Threat Landscape, 2021).

2. Hálózati szintű támadási felületek

A hálózati kommunikáció az IoT rendszerek egyik legkritikusabb és legkönnyebben támadható komponense.

A hálózati támadások tipikus kategóriái:

• Man-in-the-Middle (MITM) támadások

Gyenge vagy hiányzó TLS/DTLS implementáció esetén a támadó képes:

- üzenetek módosítására,
- lehallgatásra,
- hitelesítési információk megszerzésére.

• Replay és spoofing támadások

UDP alapú protokolloknál különösen gyakori (pl. CoAP), mivel hiányzik a kapcsolat-állapot és csomagsorrend kezelése.

• Denial-of-Service (DoS) és Distributed DoS

Az IoT eszközök alacsony erőforrásai miatt már kis mennyiségű terhelés is szolgáltatás-kiesést okozhat.

A Mirai-botnet támadás (2016) bizonyította, hogy a kompromittált IoT-eszközök tömege extrém méretű DDoS támadásokat képes generálni (Antonakakis et al., 2017).

3. Alkalmazási szintű támadási felületek

Az IoT alkalmazási rétegben gyakoriak a biztonsági hiányosságok, különösen rosszul implementált API-k és webes interfészek esetén.

Jellemző sebezhetőségek:

• Insecure API implementáció

- hitelesítetlen REST hívások
- túl széles jogosultsági körök
- sérülékeny JSON/WebSocket kommunikáció

• Input-validálási hibák

Gyakoriak:

- parancsbeillesztés (command injection),
- JSON injection,
- protokollszerű buffer overflow hibák.

• Hibás session-kezelés

Sok eszköz időkorlát nélküli session tokeneket használ ez kritikus hiba.

4. Felhő- és backend szintű támadási felületek

A modern IoT architektúrák jelentős része felhőszolgáltatásokra épül. Ez új támadási kategóriákat nyit:

• API endpointok kompromittálása

Pl. távoli eszközadminisztráció
→ ha sérülékeny, a támadó tömegesen átveheti eszközök irányítását.

• Adatszivárgás (data breach)

Egészségügyi IoT rendszerek esetén különösen kritikus (HIPAA-rendszerek, EHR adatbázisok).

• Privilege escalation felhőoldalon

A rosszul konfigurált IAM rendszer lehetővé teszi:

- eszközök újracsatlakoztatását,
- shadow device létrehozását,
- valós és hamis eszközök összekeverését.

3.3. Összegzés

Az IoT támadási felületei jelentősen kiterjedtebbek, mint a hagyományos IT-rendszereké, mivel a támadók több rétegre is hatással lehetnek: az eszközszinttől kezdve a hálózati és alkalmazási rétegen át egészen a felhőinfrastruktúráig. A biztonság több szinten történő megerősítése elengedhetetlen, ami indokolja a dolgozat későbbi fejezeteiben vizsgált TLS/DTLS/OSCORE és Zero Trust alapú védelmi modellek alkalmazását.

4. Biztonságos protokollok és védelmi mechanizmusok

Az előző fejezetekben bemutatott IoT-specifikus támadási felületek és sebezhetőségek egyértelművé teszik, hogy a hálózati kommunikáció biztonságos kialakítása kulcsszerepet játszik az IoT-rendszerek védelmében. A biztonságos protokollok feladata, hogy biztosítsák az adatok bizalmasságát, integritását és hitelességét olyan környezetben, ahol az eszközök erőforrásai korlátozottak, a hálózat heterogén, és a fizikai hozzáférés gyakran nehezen kontrollálható.

Ebben a fejezetben elsősorban a **TLS 1.3** és az **MQTT feletti biztonságos kommunikáció** kerül fókuszba, mivel a dolgozat gyakorlati része is ezen technológiákra épül. A cél annak bemutatása, hogy a TLS 1.3 milyen kriptográfiai garanciákat nyújt IoT-környezetben, illetve hogyan építhető fel rá egy biztonságos MQTT-alapú kommunikációs architektúra, amely később a Zero Trust elvekre épülő védelmi modell alapját képezheti.

4.1 A TLS 1.3 szerepe és alkalmazása IoT környezetben

A Transport Layer Security (TLS) protokoll a modern internetes kommunikáció, de facto szabványa a csatornaszintű titkosítás biztosítására. A **TLS 1.3** a protokoll legújabb, 2018-ban szabványosított verziója, amely számos olyan fejlesztést tartalmaz, amelyek különösen relevánsak az IoT-rendszerek szempontjából: egyszerűsített handshake, csökkentett késleltetés, korszerű kriptográfiai primitívek, valamint a sérülékeny, elavult algoritmusok eltávolítása.

4.1.1 A TLS 1.3 protokoll főbb sajátosságai

A TLS 1.3 egyik legfontosabb újítása, hogy **radikálisan csökkenti a kézfogás (handshake) körök számát**. A korábbi verziókhoz képest az első titkosított alkalmazásrétegbeli adatcsomag már **egy RTT (round-trip time)** után küldhető, míg bizonyos esetekben előzetesen megosztott kulcs (PSK) és session resumption esetén akár **0-RTT** adatküldés is lehetséges. Ez különösen előnyös nagy késleltetésű vagy instabil hálózatokban, amelyek az IoT-rendszerekre jellemzőek.

A protokoll kizárólag modern, úgynevezett **AEAD (Authenticated Encryption with Associated Data)** típusú titkosító algoritmusokat (pl. AES-GCM, ChaCha20-Poly1305)

engedélyez, és kötelezővé teszi a **Perfect Forward Secrecy (PFS)** használatát az ephemeral Diffie–Hellman kulcscsere révén. Ennek eredményeként egy később kompromittálódó hosszú távú kulcs önmagában nem elegendő a korábbi kommunikáció visszafejtéséhez ami kritikus követelmény érzékeny ipari vagy egészségügyi IoT-alkalmazásoknál.

A TLS 1.3 a protokoll komplexitását is csökkenti: számos elavult funkció (pl. RSA-alapú kulcscsere, statikus Diffie–Hellman, rengeteg gyenge cipher suite) eltávolításra került. Ez egyszerre **növeli a biztonságot és csökkenti az implementációs hibák valószínűségét**, ami különösen fontos az IoT-gyártók sokszor korlátozott biztonsági kompetenciáit figyelembe véve.

4.1.2 TLS 1.3 IoT-eszközökön: előnyök és korlátok

IoT-környezetben a TLS 1.3 használata egyszerre jelent **biztonsági nyereséget** és **erőforrásbeli kihívást**. A protokoll kriptográfiai műveletei különösen a kulcscsere és a tanúsítvány-ellenőrzés számottevő CPU-időt és memóriát igényelnek, ami korlátozott teljesítményű mikrokontrollerek (pl. ESP32) esetében kritikus szempont.

A szakirodalom ugyanakkor rámutat, hogy a **megfelelő optimalizációkkal** (hardveres kriptográfiai gyorsítás, PSK-alapú módok, session resumption, kis méretű tanúsítványok) a TLS 1.3 jól alkalmazható erőforrás-korlátozott környezetekben is, különösen akkor, ha a kommunikáció ritkábban, de nagyobb jelentőségű adatcsomagokra korlátozódik (pl. konfiguráció, firmware-frissítés, vezérlőparancsok).

Az IoT-rendszerekben a TLS leggyakrabban **gatewayekkel vagy felhőszolgáltatásokkal** folytatott kommunikáció során jelenik meg. Tipikus minta, hogy a szenzorok vagy aktorok MQTT-kliensként egy központi brokerhez csatlakoznak, és a brokerrel TLS 1.3-alapú titkosított csatornát hoznak létre. Ebben a modellben a végpontok közötti biztonság tényleges szintjét az határozza meg, hogy:

- a kliens és a szerver **hogyan autentikálják egymást** (csak jelszó, tanúsítvány, vagy ezek kombinációja),
- milyen **hozzáférés-szabályozási szabályok (ACL-ek)** vonatkoznak az egyes topicokra,
- milyen módon történik a **kulcskezelés és tanúsítványkezelés** (lejárat, visszavonás, rotáció).

4.1.3 TLS 1.3 és Zero Trust elvek kapcsolata

A Zero Trust modell „never trust, always verify” lényege, hogy a hálózaton belül sincs implicit bizalom, minden kapcsolatot és kérést folyamatosan hitelesíteni és autorizálni kell. A TLS 1.3 ebben a keretben **alapvető építőkö**, mivel:

- biztosítja, hogy a kommunikáció csak **hitelesített, titkosított csatornákon** történjen,
- lehetővé teszi a **kliensoldali tanúsítványok** használatát, ami az eszközidentitás erős kötését jelenti,
- támogatja az **alkalmazás szintű policy-k** kiépítését (pl. mely eszköz mely topicokra csatlakozhat).
- A dolgozat gyakorlati részében bemutatott szimulációkban a TLS 1.3-at olyan módon alkalmazom, hogy az illeszkedjen egy Zero Trust jellegű IoT-architektúra logikájához. A ténylegesen megvalósított mérések a titkosítatlan baseline és a TLS-sel védett MQTT-kommunikáció összehasonlítására fókuszálnak, míg a kölcsönös hitelesítésre (mTLS), a szigorú hozzáférés-szabályozásra és a finomhangolt policy-alapú védelemre épülő megoldások a dolgozatban elsősorban koncepcionális továbbfejlesztési irányként jelennek meg.

4.2 Biztonságos MQTT kommunikáció TLS 1.3 felett

Az MQTT egy **publish–subscribe modellre épülő, könnyűsúlyú üzenetküldő protokoll**, amelyet kifejezetten erőforrás-korlátozott eszközökre és megbízhatatlan hálózatokra terveztek. Alacsony overheadje és egyszerű implementációja miatt napjaink egyik legelterjedtebb IoT-kommunikációs megoldása, mind fogyasztói, mind ipari környezetben. Ugyanakkor natív formájában az MQTT **nem nyújt beépített titkosítást vagy hitelesítést**, ezért a biztonságot külső mechanizmusok elsősorban TLS biztosítják.

4.2.1 Az MQTT biztonsági modellje és támadási felületei

Az MQTT architektúrájának központi eleme a **broker**, amely közvetítőként működik a kliensek között. A kliensek különböző topicokra *publish*-olnak üzeneteket, illetve *subscribe*-olnak az őket érdeklő topicokra. Ennek a modellnek a biztonsági következményei a következők:

- a broker **egyetlen hiba- és támadáspontként** viselkedik (single point of failure),
- a brokerhez való jogosulatlan hozzáférés a teljes rendszer kompromittálódásához vezethet,

- megfelelő védelem nélkül egy támadó **MITM, spoofing vagy session hijacking** támadásokat hajthat végre, illetve hamis szenzoradatokat injektálhat a rendszerbe.

Ha az MQTT forgalom titkosítatlan TCP-n, hitelesítés nélkül zajlik, akkor:

- a hálózathoz hozzáférő támadó **olvashatja és módosíthatja** a szenzoradatokat,
- egyszerű eszközökkel (pl. nyílt forráskódú MQTT-kliensekkel) bármelyik topicra **feliratkozhat vagy publikálhat**,
- terheléses támadással (**üzenet-flood**) a broker vagy az erőforrás-korlátozott kliensek működése könnyen ellehetetleníthető.

Ezek a kockázatok indokolják, hogy az MQTT kommunikációt **kötelező jelleggel biztonságos csatornára** helyezzük, és szigorú hozzáférés-szabályozást alkalmazzunk.

4.2.2 MQTT over TLS 1.3: csatornaszintű védelem

A gyakorlatban az MQTT-t tipikusan két módon használják:

1. **Titkosítatlan MQTT (TCP/1883)** kizárólag zárt, erősen kontrollált hálózatokban megengedhető,
2. **MQTT over TLS (TCP/8883)** publikus vagy részben bizalmatlan hálózati környezetben ajánlott (de facto iparági standard).

Az MQTT over TLS esetén a broker **TLS-szerverként** viselkedik, a kliensek pedig TLS-kliensekként csatlakoznak hozzá. A TLS 1.3 segítségével a következő biztonsági célok érhetők el:

- **Bizalmasság:** a szenzoradatok titkosított formában utaznak a hálózaton,
- **Integritás:** az üzenetek módosítása útközben detektálható,
- **Szerverhitelesítés:** a kliens ellenőrzi, hogy valóban a megbízható brokerhez csatlakozik,
- **Opcióként klienshitelesítés:** a broker csak olyan eszközöket fogad el, amelyek érvényes tanúsítvánnyal vagy hitelesítő adatokkal rendelkeznek.

A dolgozatban tárgyalt szimulációs környezet három tipikus konfigurációt különít el:

- **(1) MQTT titkosítás és hitelesítés nélkül:** baseline, magas kockázatú konfiguráció,

- **(2) MQTT TLS 1.3 felett, jelszavas hitelesítéssel:** köztes szint, csatornaszintű védelem, de gyengébb eszköz-identitás,
- **(3) MQTT TLS 1.3 felett, kölcsönös tanúsítványalapú hitelesítéssel (mTLS) és ACL-ekkel:** erős eszközidentitás, finomhangolt hozzáférés-szabályozás a Zero Trust architektúra irányába mutató megoldás.

A későbbi mérések ezen három konfiguráció erőforrás-igényét, késleltetését és biztonsági szintjét hasonlítják össze.

4.2.3 Hozzáférés-szabályozás és topic-szintű védelem

A TLS 1.3 önmagában a csatorna biztonságát garantálja, de **nem dönt arról**, hogy egy adott eszköz milyen adatokhoz férhet hozzá. Ezt az MQTT-ben a broker szintjén működő **hozzáférés-szabályozási listák (Access Control List – ACL)** valósítják meg.

Egy tipikus IoT-szenzorhálózatban például az alábbi policy-k definiálhatók:

- az adott szenzor **csak egy konkrét topicra** (pl. szenzor/hu/gyar1/homerseklet) publikálhat,
- nem iratkozhat fel más eszközök topicjaira,
- a vezérlő vagy „felhőalkalmazás” csak olvasási jogosultságot kap a szenzor-topicokra, míg vezérlő topicokra (pl. actuator/...) csak meghatározott komponensek írhatnak.

Az ACL-ek és a TLS-es eszköz-hitelesítés kombinációja lehetővé teszi, hogy a rendszer **identity-aware** módon döntsön a hozzáférésekről, vagyis ne IP-cím, hanem **tanúsítványhoz vagy felhasználói fiókhoz kötött identitás** alapján. Ez a Zero Trust modell egyik kulcseleme, amelyet a dolgozat 6. fejezete részletesen tárgyal.

4.2.4 Kapcsolódás a dolgozat gyakorlati részéhez

A dolgozat 5. fejezetében bemutatott szimulációs környezet központi eleme egy MQTT-brokerrel kommunikáló, szimulált IoT-eszközökből álló rendszer. A Wokwi-alapú ESP32-szimulációk és a TLS 1.3-at támogató broker konfigurációi lehetővé teszik:

- a titkosítatlan és a titkosított MQTT-forgalom **késleltetési és overhead különbségeinek** mérését,
- a jelszavas és tanúsítványalapú hitelesítés **összehasonlítását**,
- egyszerű támadási forgatókönyvek (jogosulatlan publish, flood jellegű terhelés) hatásának vizsgálatát a különböző védelmi szintek mellett.

Ezzel a 4.2 fejezetben bemutatott elméleti keret közvetlenül átvezet a dolgozat empirikus részéhez, és megalapozza a későbbi teljesítmény- és biztonsági összehasonlításokat.

4.3 Zero Trust alapelvek és alkalmazásuk IoT környezetben

Az előző fejezetekben bemutatott támadási felületek és incidensek azt mutatják, hogy a klasszikus, peremvédelemre épülő biztonsági modell amely a belső hálózatot „megbízható”, a külvilágot pedig „nem megbízható” zónaként kezeli az IoT-rendszerek esetén nem bizonyul elegendőnek. Az eszközök nagy száma, heterogenitása, földrajzi szétszórtsága és a felhőszolgáltatásokra támaszkodó architektúrák miatt a hálózati határok elmosódnak, és gyakorlatilag bármely komponens potenciális belépési ponttá válhat egy támadó számára. Erre a kihívásra válaszul jelent meg a **Zero Trust** biztonsági modell, amely az „implicit bizalom” helyett a folyamatos ellenőrzésre és a minimális jogosultság elvére épít. (NIST, 2020; ENISA, 2023).

4.3.1 A Zero Trust modell fogalma és alapelvei

A Zero Trust lényege röviden úgy foglалható össze, hogy **„soha ne bízz meg alapértelmezésben, mindig ellenőrizz”**. A modell szerint sem a belső hálózaton lévő eszközök, sem a felhasználók, sem az alkalmazások nem tekinthetők automatikusan megbízhatónak pusztán a hálózati elhelyezkedésük alapján. Ehelyett minden kérésnél vizsgálni kell az identitást, a kontextust és az aktuális kockázati szintet. (NIST, 2020).

A Zero Trust szakirodalma több, egymást kiegészítő alapelvet fogalmaz meg (NIST, 2020; Microsoft, 2021):

- **Identitás-központú védelem:** a védelmi döntések központi eleme az eszköz, felhasználó vagy szolgáltatás identitása, nem pedig az IP-cím vagy a hálózati szegmens.
- **Legkisebb jogosultság elve (least privilege):** minden entitás csak a működéséhez feltétlenül szükséges erőforrásokhoz kap hozzáférést, a lehető legrövidebb ideig.
- **Mikroszegmentáció:** a hálózat és az alkalmazások logikai felosztása kisebb, izolált zónákra, hogy egy esetleges kompromittáció ne tudjon kontrollálatlanul elterjedni.
- **Folyamatos monitorozás és verifikáció:** a hozzáférési döntések nem egyszeri autentikáció alapján születnek, hanem folyamatosan figyelembe veszik az eszköz állapotát, a viselkedést és az aktuális fenyegetettségi szintet.

Ezek az elvek jól illeszkednek a dolgozat kutatási céljához, amely egy Zero Trust-alapú, többretegű védelmi architektúra kidolgozását tűzi ki célul IoT-rendszerekre.

4.3.2 Zero Trust IoT-rendszerekben: identitás, eszközállapot és hálózati szegmentáció

IoT környezetben a Zero Trust alkalmazása több, egymásra épülő szinten jelenik meg:

1. Eszközidentitás és hitelesítés

Az IoT-eszközök esetében alapvető követelmény, hogy minden eszköz egyértelműen azonosítható legyen, és az azonosítás kriptográfiailag legyen védve. Ennek gyakori megoldása a **tanúsítványalapú eszközidentitás** (X.509 tanúsítványok, mTLS), amelyet a dolgozat gyakorlati része is alkalmaz az MQTT-kliens és a broker közötti kommunikációban. A hardcoded jelszavak, közös default hitelesítési adatok használata ezzel szemben Zero Trust szempontból elfogadhatatlan, amit az olyan ajánlások is kiemelnek, mint az ETSI EN 303 645.

2. Eszközállapot és megfelelés (posture)

A Zero Trust modell az identitáson túl az eszköz aktuális **biztonsági állapotát** is figyelembe veszi: friss-e a firmware, engedélyezve van-e a secure boot, érvényesek-e a tanúsítványok, megfelel-e a konfiguráció a szervezeti policy-knak stb. IoT-ben ez különösen fontos, mert sok eszköz évekig felügyelet nélkül üzemel, és könnyen elavulttá válhat. (NISTIR 8259, 2020; ENISA, 2023).

3. Hálózati mikroszegmentáció és broker-központú kontroll

Mivel az IoT-eszközök gyakran gatewayeken, brokerekén vagy edge node-okon keresztül érik el egymást, a Zero Trust által javasolt mikroszegmentáció IoT-ben a következőt jelenti:

- az eszközök csak **szűk körű, jól definiált szolgáltatásokhoz** férhetnek hozzá (pl. egyetlen MQTT brokerhez, meghatározott topicokra),
- a broker vagy gateway **policy-alapú döntéseket** hoz arról, hogy az adott identitás milyen műveleteket hajthat végre,
- a hálózati szinten tűzfalak, VLAN-ok, VPN-ek vagy SDN-alapú szegmensek szűkítik a lehetséges laterális mozgást.

A dolgozatban bemutatott szimulációkban ez a szemlélet úgy jelenik meg, hogy az MQTT-broker TLS 1.3 és mTLS használatával a tanúsítványhoz kötött identitás alapján dönti el, hogy az egyes eszközök mely topicokra publish-olhatnak vagy iratkozhatnak fel.

4.3.3 Zero Trust és defense-in-depth IoT környezetben

A Zero Trust modell önmagában nem váltja ki a hagyományos **többrétegű védelem** (defense-in-depth) elvét, hanem azzal kombinálva nyújt átfogó védelmet. A H3 hipotézis is azt állítja, hogy a Zero Trust és a defense-in-depth együttes alkalmazása hatékony védelmet kínál a fejlett, perzisztens fenyegetésekkel szemben.

IoT-rendszerben ez a gyakorlatban például a következő rétegeket jelenti:

- **Eszközsztintű védelem:** secure boot, titkosított tárhely, hardveres kulcstárolás (TPM, secure element), biztonságos frissítési mechanizmusok.
- **Hálózati réteg védelme:** TLS/DTLS alapú titkosítás, szigorú tűzfalszabályok, hálózati szegmensek közötti forgalom korlátozása.
- **Alkalmazási és API-sztintű kontroll:** erős autentikáció, jogosultságkezelés, input-validáció, rate limiting.
- **Monitorozás és incidenskezelés:** naplózás, SIEM-integráció, anomáliadetektáló rendszerek, automatizált riasztások és reakciók (pl. gyanús eszköz kvázi-azonnali izolálása).

A Zero Trust ebben a keretben úgy értelmezhető, mint **irányelv-rendszer**, amely meghatározza, hogy a fenti rétegek hogyan viselkedjenek: ne bízzanak sem az eszközben, sem a felhasználóban, sem a hálózati helyzetben önmagában, hanem mindig kombinált, kontextusfüggő döntéseket hozzanak.

4.3.4 Kapcsolódás a dolgozat kutatási kérdéseivel és gyakorlati részéhez

A dolgozat bevezető fejezetében megfogalmazott kutatási kérdések között szerepel, hogy **miként implementálható a Zero Trust modell IoT környezetben**, illetve hogyan építhető rá egy többrétegű védelmi mechanizmus. A 4.3 fejezetben bemutatott elméleti keret közvetlenül megalapozza a későbbi gyakorlati munkát:

- a Wokwi-alapú ESP32-szimulációkban az eszközidentitás és a TLS 1.3/mTLS használata a Zero Trust „identity-centric” pillérének felel meg;
- az MQTT-broker ACL-jei és a topic-sztintű policy-k a mikroszegmentáció és a minimális jogosultság elvét valósítják meg;
- a naplózás, a Wireshark/tcpdump-alapú forgalomelemzés és az anomáliák vizsgálata a folyamatos monitorozás gyakorlati eszközei.

A 6. fejezetben kidolgozásra kerülő integrált biztonsági keretrendszer már kifejezetten Zero Trust szemléletben épül fel: az IoT-eszközök, a hálózati infrastruktúra és a felhőszolgáltatások úgy kapcsolódnak egymáshoz, hogy minden hozzáférés explicit ellenőrzésen és policy-alapú döntéseken keresztül valósul meg. Ezzel a dolgozat nemcsak elméleti szinten tárgyalja a Zero Trust modellt, hanem egy olyan, IoT-specifikus implementációs mintát is bemutat, amely gyakorlati referenciaként szolgálhat fejlesztők és rendszertervezők számára.

4.4 A vizsgált biztonsági megoldások és a szimulációs kutatás kapcsolata

A dolgozat bevezető fejezetében megfogalmazott fő kutatási kérdés arra irányul, hogy **milyen sebezhetőségek jellemzik leginkább az IoT-eszközöket, és ezek ellen milyen hálózati szintű védelmi mechanizmusok és architektúrák képesek hatékony megoldást nyújtani**. Ehhez kapcsolódnak az alcélok és hipotézisek, amelyek külön-külön vizsgálják a biztonságos protokollok, a Zero Trust modell és az automatizált védelem szerepét.

A 2. és 3. fejezet elsősorban a **technológiai háttér** és az **IoT-specifikus támadási vektorok** bemutatásával foglalkozott, kiemelve, hogy a hálózati kommunikáció különösen az MQTT és a CoAP kulcsfontosságú támadási felület. A 4. fejezet ehhez kapcsolódva már kifejezetten a **védelmi mechanizmusokra**, azon belül is a TLS 1.3-ra, a biztonságos MQTT-kommunikációra és a Zero Trust szemléletre koncentrált.

A következő, 5. fejezetben a hangsúly az elméleti keretokről a **gyakorlati, szimulációs vizsgálatokra** helyeződik át. Ennek célja, hogy empirikus mérések segítségével értékelje a 4. fejezetben tárgyalt megoldások hatását olyan szempontok mentén, mint a késleltetés, az erőforrás-igény és a támadásokkal szembeni ellenálló képesség. A 4.4 fejezet feladata ezért az, hogy expliciten megmutassa, hogyan „fordítódnak le” az elméleti fogalmak (TLS 1.3, MQTT over TLS, Zero Trust, defense-in-depth) a szimulációs környezet konkrét beállításaira és mérési szcenárióira.

4.4.1 Kapcsolódás a kutatási kérdésekhez és hipotézisekhez

A dolgozat elején megfogalmazott hipotézisek (H1–H4) a 3–6. fejezeteken keresztül kapnak választ. Röviden:

- **H1** (támadási felületek és tipikus sebezhetőségek) elsősorban a 3. fejezetben kerül bizonyításra, ahol konkrét sérülékenységtípusok és valós támadási esetek (pl. Mirai-botnet, Realtek sebezhetőségek) bemutatásán keresztül látható, hogy a gyenge hitelesítés, az alapértelmezett jelszavak és az elavult protokollok milyen kockázatot jelentenek. A szimulációban ennek „laboratóriumi megfelelője” az a konfiguráció, ahol

az MQTT-broker titkosítás és hitelesítés nélkül érhető el, és ahol a támadó kliens jogosulatlanul tud üzeneteket publikálni vagy feliratkozni.

- **H2** a modern biztonságos protokollok különösen a TLS 1.3 IoT-környezetben való alkalmazhatóságára és overheadjére fókuszál. A 4.1 és 4.2 alfejezet bemutatta a TLS 1.3 technikai sajátosságait és az MQTT over TLS modell előnyeit, míg az 5. fejezet mérései azt vizsgálják, hogy a titkosított kommunikáció mekkora többlet-késleltetést és erőforrás-igényt jelent az IoT-eszközök számára a titkosítatlan baseline-hoz képest.
- **H3** a Zero Trust modell és a defense-in-depth kombinációjának IoT-ra adaptált hatékonyságát vizsgálja. A 4.3 fejezet elméleti szinten mutatta be a Zero Trust alapelveit, míg a 6. fejezetben egy olyan architektúra-javaslat kerül kidolgozásra, amely a szimulációban tesztelt mechanizmusokra (mTLS, ACL-ek, topic-szintű policy-k, monitorozás) épül. A 4. és 5. fejezet így együtt készítik elő a 6. fejezetben bemutatott keretrendszer validálását.
- **H4** az automatizált sebezhetőség-felismerés és anomáliadetekció szerepét emeli ki. Bár a jelen dolgozat fókusza elsősorban a protokoll-szintű védelemre és az architektúrára helyeződik, a 6. fejezetben vázolt biztonsági keretrendszer koncepcionális szinten számol azzal, hogy a naplózott MQTT-forgalomra később MI-alapú anomáliadetekciós modul kapcsolható.

Ezzel a 4. fejezet és benne a 4.4 alfejezet világosan összeköti a **kutatási kérdéseket**, a **választott technológiákat** és a **gyakorlati mérések logikáját**.

4.4.2 A biztonsági megoldások leképezése a szimulációs környezetre

A 2.3–2.4 fejezetekben részletesen indoklásra került, hogy a vizsgálat szempontjából az MQTT és a CoAP protokollok, valamint a TLS 1.3, a DTLS és az OSCORE jelentik a legrelevánsabb biztonsági technológiákat. A gyakorlati részben azonban a dolgozat terjedelmi és időbeli korlátai miatt egy **szűkebb, de mélyebben vizsgált részhalmazra** fókuszálunk:

- a Wokwi-alapú szimulációk középpontjában **MQTT kliensként működő ESP32-eszközök** állnak,
- a kommunikáció biztonságát **TLS 1.3** biztosítja,
- a Zero Trust alapelvek eszközszinten **mTLS** és **topic-szintű ACL-ek** formájában jelennek meg.

Ez a választás több szempontból is indokolt:

1. Az MQTT TCP-alapú, brokerközpontú modellje jól illeszkedik a TLS 1.3-hoz, és iparági szinten bevett gyakorlat az MQTT over TLS használata.
2. A broker központi szerepe miatt az MQTT-architektúra kiválóan alkalmas annak demonstrálására, hogy a hibásan konfigurált hitelesítés, a gyenge hozzáférés-szabályozás és a hiányzó titkosítás hogyan növeli a támadási felületet és hogyan csökkenthető ez a megfelelő védelmi mechanizmusokkal.
3. A TLS 1.3 overheadjének mérése erőforrás-korlátozott eszközökön (ESP32) közvetlenül választ ad a H2 hipotézisben megfogalmazott kérdésre.

A 4. fejezetben bemutatott elméleti elemek tehát konkrétan így jelennek meg az 5. fejezet szimulációs környezetében:

- **TLS 1.3** → a Wokwi-s ESP32-k és a Mosquitto broker közötti titkosított csatorna,
- **MQTT biztonsági modellje** → különböző brokerkonfigurációk (nyílt, jelszavas, mTLS + ACL),
- **Zero Trust elvek** → eszközidentitás tanúsítvánnyal, minimális jogosultság, szigorú topic-policy-k, naplózás.

Az így kialakított laboratóriumi környezetben az 5. fejezet mérései meg tudják mutatni, hogy **ugyanazon IoT-alkalmazás** hogyan viselkedik biztonsági szempontból három különböző konfigurációban (nincs védelem, TLS + jelszó, TLS + mTLS + ACL).

4.4.3 Korlátok és további bővítési lehetőségek

A 1.6 fejezetben részletesen ismertetett kutatási korlátok alapján a gyakorlati vizsgálatok **laboratóriumi, szimulált környezetben** zajlanak, korlátozott méretű eszközparkkal, és nem terjednek ki több tízezer eszközt érintő skálázhatósági tesztekre. Ennek megfelelően:

- a CoAP/DTLS/OSCORE protokollok a dolgozatban elsősorban **elméleti és koncepcionális szinten** jelennek meg (2.4 és 4. fejezet),
- a gyakorlati mérések szűkebb fókuszban, **MQTT + TLS 1.3** környezetben vizsgálják a biztonság és teljesítmény kompromisszumát,
- a szimuláció célja nem a teljes IoT-ökoszisztéma lefedése, hanem **reprezentatív mintán** keresztül a főbb trendek és összefüggések bemutatása.

Ugyanakkor a kiépített szimulációs környezet és a 6. fejezetben bemutatott architektúra alapul szolgálhat **további bővített vizsgálatokhoz** is. A későbbi kutatás például:

- kiterjesztheti a méréseket CoAP + DTLS/OSCORE környezetre,
- bevonhat valódi fizikai eszközöket a szimuláció mellett,
- MI-alapú anomáliadetekciós modulokkal egészítheti ki az MQTT-forgalom monitorozását.

Ezzel a 4. fejezet lezárása egyértelműen kijelöli az 5. fejezet feladatát: a korábban ismertetett elméleti modellek és védelmi mechanizmusok **empirikus vizsgálatát** egy jól definiált, IoT-re jellemző szimulációs környezetben.

5. A szimulációs környezet és mérési módszertan

5.1 A szimulációs környezet felépítése

Az 5. fejezet célja, hogy a 2–4. fejezetekben bemutatott elméleti kereteket gyakorlati, mérhető formában is vizsgálja. Ehhez egy olyan, jól reprodukálható laboratóriumi környezet készült, amelyben szimulált IoT-eszközök MQTT-n keresztül kommunikálnak egy brokerrel, a forgalmat pedig Python-alapú backend komponensek gyűjtik és elemzik. A környezet kifejezetten arra lett kialakítva, hogy:

- összehasonlítható legyen a titkosítatlan és a TLS 1.3-mal védett MQTT-kommunikáció,
- láthatóvá váljanak a különböző biztonsági szintek (nyitott, jelszavas, mTLS + ACL) hatásai,
- kontrollált módon legyenek szimulálhatók egyszerű támadási forgatókönyvek (jogosulatlan publish, túlterhelés jellegű forgalom).

5.1.1 A környezet fő komponensei

A szimulációs rendszer logikai szinten négy fő szereplőből áll:

1. Szimulált IoT-eszközök (szenzorok)

- Két darab ESP32 fejlesztőpanel szimulációja Wokwi környezetben („sensor1” és „sensor2” projekt).
- Mindkét eszköz Wi-Fi kapcsolaton keresztül csatlakozik az internethez, és MQTT-kliensként viselkedik.
- A szenzorok 5 másodperces periódussal hőmérsékleti mérésnek tekintett, véletlenszerűen generált értékeket publikálnak.

2. MQTT-broker

- A kezdeti baseline mérésekhez egy nyilvános, titkosítatlan MQTT-broker (TCP/1883) kerül felhasználásra.
- A további vizsgálatokhoz lokálisan futó Mosquitto broker szolgál, két konfigurációval:
 - titkosítatlan MQTT (1883),
 - TLS 1.3-mal védett MQTT (8883), saját CA által aláírt szervertanúsítvánnyal.

3. Mérési backend (collector)

- Pythonban írt, paho-mqtt klienskönyvtárra épülő adatgyűjtő komponens (5.2 alfejezet).
- Feladata a szenzor-topicokra való feliratkozás, az üzenetek fogadása és CSV-fájlba történő rögzítése.

4. Elemző modul

- A measurements.csv fájlt feldolgozó Python-szkript (analyze_measurements.py), amely szenzoronként kiszámítja:
 - az üzenetek számát,
 - a mérések időtartamát,
 - az üzenetküldési rátát,
 - az átlagos hőmérsékletet.
- Ezek a mutatók képezik a titkosítatlan és titkosított konfigurációk összehasonlításának alapját.

A komponensek közötti kapcsolatot egyszerű, „csillag topológiájú” elrendezés valósítja meg: minden szenzor a brokerhez kapcsolódik, a backend pedig „megfigyelőként” iratkozik fel ugyanazokra a topicokra.

5.1.2 Hardver- és szoftverkörnyezet

A szimulációs környezet kialakítása során a következő eszközök és szoftverek kerültek felhasználásra:

- **Fejlesztői munkaállomás**

- Operációs rendszer: Windows-alapú kliensgép.
- Fő szerepe: a Mosquitto broker és a Python backend futtatása, valamint az eredmények feldolgozása.
- **IoT-eszköz szimuláció: Wokwi + ESP32**
 - A Wokwi online szimulátorban két külön projekt készült: sensor1 és sensor2.
 - Mindkettő az ESP32 DevKitC v4 fejlesztőpanel virtuális példányát használja.
 - A kód Arduino-stílusú C/C++ (sketch.ino), amely a WiFi- és az MQTT-klienseket inicializálja, majd periodikusan publikálja a méréseket.
- **MQTT-broker: Mosquitto**
 - A Mosquitto telepítése után két konfigurációs állomány készült:
 - alapértelmezett, titkosítatlan konfiguráció,
 - TLS-t támogató konfiguráció, amely:
 - 8883-as porton fogadja az MQTT-over-TLS kapcsolatokat,
 - betölti a saját CA-tanúsítványt (ca.crt),
 - szerver tanúsítványt és kulcsot (server.crt, server.key).
- **Python környezet**
 - Python 3.9 interpreter.
 - Főbb csomagok:
 - paho-mqtt – MQTT kliensfunkciók (collector modulok),
 - beépített csv, json és datetime a mért adatok kezeléséhez és időbélyegek formázásához.

Ez a környezet könnyen reprodukálható: az ESP32 kódja, a broker konfigurációja és a Python szkriptek egyaránt verziózott fájlok, így a mérések később azonos beállításokkal újra lefuttathatók.

5.1.3 Hálózati topológia és MQTT-topic struktúra

A szimulációs hálózatban az eszközök az alábbi logikai elrendezés szerint kapcsolódnak:

- mindkét ESP32 kliens (sensor1, sensor2) MQTT-kapcsolatot nyit a broker felé,

- a broker egyetlen központi csomópontként továbbítja az üzeneteket a feliratkozott klienseknek,
- a Python backend ugyanarra a brokerre csatlakozik, és megfigyelőként viselkedik.

A mérések során használt topicok:

- `iot/lab/sensor1/temperature`
- `iot/lab/sensor2/temperature`

Ez a struktúra egyszerű, mégis jól példázza az MQTT-ben megszokott hierarchikus elnevezési sémát. Szükség esetén a későbbi bővítések (pl. újabb szenzorok, egyéb metrikák) további altopicok definiálásával könnyen integrálhatók a rendszerbe.

Az egyes üzenetek payloadja JSON formátumú, egységes mezőstruktúrával:

V.I.I. ábra – Az MQTT üzenet JSON formátumú payloadja

```
{
  "sensor_id": "sensor1" | "sensor2",
  "ts_device": <eszköz oldali időbélyeg milliszekundumban>,
  "temperature": <egész szám, Celsius-fok>
}
```

Forrás: saját szerkesztés

Ezzel biztosítható, hogy a backend a különböző szenzoroktól érkező üzeneteket egységes módon tudja feldolgozni, és az elemző modul szenzorazonosító alapján csoportosíthassa az adatokat.

5.1.4 A mérési folyamat lépései

A tényleges mérési futások minden esetben ugyanazon alaplépések szerint zajlanak, hogy az eredmények összehasonlíthatók legyenek:

1. Broker inicializálása

- Baseline futásnál a titkosítatlan broker-konfiguráció indul (1883).
- Biztonságos futásnál a TLS-es Mosquitto konfiguráció indul (8883, saját CA-val).

2. Backend indítása

- A megfelelő collector modul (titkosítatlan vagy TLS-es) elindul, feliratkozik a szenzor-topicokra, és létrehozza a measurements.csv állományt.
- A program a futás során minden üzenetet időbélyeggel együtt rögzít.

3. Szenzorok indítása Wokwi-ban

- A sensor1 és sensor2 projektek szimulációja elindul.
- A szenzorok csatlakoznak a WiFi-hálózathoz, majd MQTT-kapcsolatot hoznak létre a brokerrel.
- 5 másodperces periódussal hőmérsékleti értékeket publikálnak a saját topicjukra.

4. Mérés hossza

- Egy mérési futás tipikusan néhány percre tart; ez elegendő adatpontot szolgáltat az üzenetküldési ráta és az átlaghőmérséklet stabil becsléséhez.
- A titkosítatlan és a TLS-es futások hasonló, néhány perces időtartamban zajlanak, így az üzenetküldési ráták és átlaghőmérsékletek összehasonlíthatók.

5. Adatgyűjtés lezárása és elemzés

- A futás végén a backend leáll, a measurements.csv fájl lezárul.
- Az elemző szkript (analyze_measurements.py) feldolgozza a fájlt, és szenzoronként kiszámítja a fő statisztikákat (5.3 alfejezet).

Ezzel a módszerrel a szimulációs környezet egyszerre egyszerű és kellően kontrollált: a mért különbségek a kommunikációs csatorna biztonsági konfigurációjából és a broker beállításából adódnak, miközben a szenzorlogika és a mintavételezési periódus változatlan marad.

5.2 MQTT-alapú mérési backend megvalósítása

Az előző alfejezetekben bemutatott szimulációs környezetben a Wokwi-ban futó ESP32-es szenzorok MQTT-n keresztül kommunikálnak a brokerrel. Ahhoz, hogy a mérések kiértékelhetők legyenek, szükség van egy olyan backend komponensre, amely:

- feliratkozik a szenzorok által használt topicokra,
- minden beérkező üzenetet időbélyeggel együtt rögzít,

- az adatokat később statisztikai elemzésre alkalmas formában (CSV) elérhetővé teszi.

Ezt a szerepet a Pythonban, *paho-mqtt* könyvtárra épülő **collector** és **analyzer** modulok látják el. Az implementációt úgy alakítottam ki, hogy a titkosítatlan és a TLS-sel védett kommunikáció között a lehető legkevesebb kódbeli különbség legyen: a két collector modul belső logikája azonos, csak a broker eléréséhez használt paraméterek és a TLS-hez kapcsolódó beállítások térnek el.

5.2.1 Követelmények és szerepkör

A mérési backend tervezésekor az alábbi követelményeket vettem figyelembe:

- **Egyszerűség** és **átláthatóság:**
a kód legyen könnyen követhető, jól dokumentálható, ne használjon szükségtelenül bonyolult keretrendszereket;
- **Reprodukálhatóság:**
ugyanaz a backend legyen használható különböző brokerkonfigurációk (baseline, TLS) mellett is, minimális módosítással;
- **Formátumfüggetlenség:**
a logger ne „értelmezze túl” a payloadot, hanem a nyers JSON-t mentse el; az értelmezés a későbbi elemző modul feladata;
- **További** **bővíthetőség:**
a kialakított struktúra tegye lehetővé további szenzorok, újabb topicok vagy akár támadó kliensek méréseinek naplózását.

Ezeket a követelményeket egy egyszerű, eseményvezérelt architektúra teljesíti: az MQTT kliens callback függvények segítségével reagál a broker eseményeire (`on_connect`, `on_message`), a háttérben pedig egy folyamatosan nyitva tartott CSV-fájlba írja az adatokat.

5.2.2 Titkosítatlan collector modul (`collector.py`)

A **collector.py** a mérési backend alapváltozata, amely titkosítatlan MQTT-kapcsolaton keresztül gyűjti az adatokat. Ez szolgál baseline referenciaként a későbbi, TLS-t használó konfigurációkhoz.

A modul legfontosabb elemei:

- **Brokerparaméterek és topiclista**

A program elején néhány konstans határozza meg, hogy melyik brokerhez és mely topicokra csatlakozik a collector:

```
BROKER = "test.mosquitto.org" # vagy: "localhost"
PORT = 1883
TOPIC = "iot/lab/#"
```

Forrás: saját szerkesztés

A `iot/lab/#` wildcard topic biztosítja, hogy a collector a `iot/lab/sensor1/temperature` és `iot/lab/sensor2/temperature` topicokra egyaránt megkapja az üzeneteket, illetve a későbbi bővítések során új altopicok is lefedhetők legyenek.

- **CSV-fájl inicializálása**

A program a futás elején megnyitja (vagy létrehozza) a `measurements.csv` fájlt, és fejlécsort ír bele:

```
recv_time,topic,payload
```

Forrás: saját szerkesztés

Minden további sor egy beérkezett MQTT-üzenetnek felel meg; a mezők jelentése megegyezik az 5.1.3 alfejezetben bemutatott JSON-mintával.

- **Kapcsolódási callback (`on_connect`)**

Az `on_connect` függvény feladata, hogy sikeres kapcsolódás esetén feliratkozzon a megadott topicra:

- logolja a broker által visszaadott eredménykódot (diagnosztika),
- `client.subscribe(TOPIC)` hívással kérje a `iot/lab/#` topic-családra vonatkozó üzeneteket.

Így biztosítható, hogy a collector automatikusan újrafeliratkozik akkor is, ha a kapcsolat valamilyen hiba miatt megszakad, majd újra felépül.

- **Üzenetkezelő callback (`on_message`)**

Az `on_message` függvény minden beérkező üzenetnél meghívódik. Feladata:

- a fogadási idő (`recv_time`) rögzítése UTC-ben (`datetime.utcnow()`),
- a topicnév (`msg.topic`) és a payload (`msg.payload.decode("utf-8")`) beolvasása,

- a három érték CSV-sorba írása, majd a fájl flush-elése.

A collector nem próbálja meg feldolgozni vagy értelmezni a JSON-t; a nyers payload kerül eltárolásra, ami rugalmasságot ad a későbbi adatfeldolgozáshoz.

- **Fő programciklus**

A fő rész létrehozza az MQTT-kliens objektumot, beállítja a callbackeket, csatlakozik a brokerhez, majd elindítja a végtelen `loop_forever()` ciklust. A program addig fut, amíg a teljes mérési időablak le nem telik.

Ezzel a modul egy egyszerű, de jól skálázható baseline logger szerepét tölti be: akár valós, akár szimulált szenzorokról érkeznek az adatok, a collector program működése változatlan.

5.2.3 TLS-sel védett collector modul (`collector_tls.py`)

A `collector_tls.py` a backend TLS-képes változata. Felépítése szándékosan nagyon hasonló a titkosítatlan verzióhoz, hogy az eltérések egyértelműen visszavezethetők legyenek a biztonsági konfigurációra.

A fő különbségek:

- **Brokerbeállítás**

```
BROKER = "localhost"
PORT  = 8883
TOPIC = "iot/lab/#"
```

Forrás: saját szerkesztés

A collector a lokálisan futó Mosquitto broker TLS-es portjára csatlakozik, amelyet a saját CA (`ca.crt`) által aláírt szervertanúsítvány véd.

□ CA-tanúsítvány használata

A program a munkakönyvtárban lévő `ca.crt` fájlt adja át a *paho-mqtt* kliensnek:

```
client.tls_set(
    ca_certs="ca.crt",
    certfile=None,
    keyfile=None,
    tls_version=ssl.PROTOCOL_TLS_CLIENT,
)
```

Forrás: saját szerkesztés

Ennek hatására a kliens a TLS-kézfogás során ellenőrzi, hogy a broker által küldött tanúsítványt valóban a megadott CA írta-e alá, és hogy az érvényes-e. Így a collector csak a „valódi” brokerrel hoz létre titkosított kapcsolatot.

□ Logika és CSV-kezelés

Minden egyéb szempontból a TLS-es collector megegyezik a titkosítatlan változattal:

- ugyanazt a topic-családot figyeli,
- ugyanabba a formátumú CSV-be ír,
- ugyanazt az `on_connect` és `on_message` logikát használja.

Ez a kialakítás biztosítja, hogy a baseline (1883-as) és a TLS-es (8883-as) futások mérési eredményei közvetlenül összehasonlíthatók: ugyanaz a kód, ugyanaz a mintavételezés, csak a kommunikációs csatorna biztonsági szintje tér el.

5.2.4 Tesztpublikáló szkriptek (`pub_test.py`, `pub_test_tls.py`)

A Wokwi-szenzorok mellett készült két egyszerű „tesztkliens” is:

- **`pub_test.py`** titkosítatlan MQTT-hívásokra,
- **`pub_test_tls.py`** TLS-en keresztüli hívásokra.

Mindkét szkript ugyanazt a logikát követi:

- MQTT-klienst hoz létre,
- csatlakozik a megfelelő brokerhez (1883 vagy 8883),
- egy rövid ciklusban néhány mesterséges „hőmérsékleti mérést” publikál az `iot/lab/testsensor/temperature` topicra.

Ezek a szkriptek két célra szolgálnak:

1. **Funkcionális** **ellenőrzés:**
használatukkal gyorsan ellenőrizhető, hogy a broker-konfiguráció és a collector modul helyesen működik-e (maga a Wokwi nélkül is).
2. **Teszt- és támadási** **szcenáriók** **alapja:**
a későbbi fejezetekben ezek módosított változatai jelennek meg „támadó kliensként”, amely jogosulatlan topicokra próbál publikálni, illetve mTLS/ACL mellett elvárt módon elutasításra kerül.

5.2.5 Elemző modul (`analyze_measurements.py`)

Az adatok gyűjtése önmagában még nem elegendő, ezért készült egy külön elemző szkript, az **`analyze_measurements.py`**, amely a `measurements.csv` (illetve TLS-es futásnál `measurements_tls.csv`) tartalmát dolgozza fel. A program feladata:

- a CSV-sorok beolvasása,

- a JSON payloadok értelmezése,
- az üzenetek csoportosítása sensor_id alapján,
- szenzoronként az alábbi mutatók kiszámítása:
 - üzenetszám,
 - első és utolsó üzenet időbélyege,
 - üzenetküldési ráta (üzenet/másodperc),
 - átlagos hőmérséklet.

A szkript kimenete egy egyszerű, konzolra írt összefoglaló, például:

```
==== sensor1 ====
Üzenetek száma: 56
Időtartam: 1059.9 s
Becsült üzenetküldési ráta: 0.053 msg/s
Átlagos hőmérséklet: 24.96 °C
```

Forrás: saját szerkesztés

Ugyanez megismételhető a TLS-es futásból származó adatokra is, így a mérésekből származó különbségek a 5.3 fejezetben bemutatott módszertannal és mérőszámokkal elemezhetők, és a későbbi fejezetekben grafikonok, táblázatok formájában is ábrázolhatók.

5.3 Mérési módszertan és vizsgált scenáriók

Az 5.1 fejezetben bemutatott szimulációs környezet és az 5.2-ben ismertetett MQTT-alapú mérési backend lehetővé teszi, hogy a 4. fejezetben tárgyalt biztonsági megoldások hatását empirikus módon vizsgáljuk. A mérések célja, hogy számszerűsítsék, milyen többletterhelést és védelmi szintet jelent az MQTT kommunikáció különböző biztonsági konfigurációkban, és ezzel alátámaszák a 4.2.2 alfejezetben definiált három tipikus beállítás összehasonlítását.

5.3.1 Vizsgált brokerkonfigurációk

A mérések három, egymásra épülő biztonsági szinten történnek, amelyek a 4. fejezetben elméleti szinten már megjelentek:

1. Konfiguráció A titkosítatlan MQTT, hitelesítés nélkül (baseline)

- Kommunikáció: plain MQTT, TCP/1883, TLS nélkül.
- Hitelesítés: nincs; bármely kliens csatlakozhat a brokerhez és tetszőleges topicra publikálhat/feliratkozhat.
- Cél: referenciapont, amely megmutatja a „legolcsóbb”, de leginkább sebezhető működési módot.

2. Konfiguráció B MQTT TLS 1.3 felett, szerverhitelesítéssel

- Kommunikáció: MQTT over TLS 1.3, TCP/8883.
- Hitelesítés: a broker X.509 szervertanúsítvánnyal azonosítja magát; a kliensek a saját CA-tanúsítvány alapján ellenőrzik a szerver identitását.
- Cél: a csatornaszintű védelem (bizalmasság, integritás, szerverhitelesítés) teljesítményhatásainak vizsgálata a baseline-hoz képest.

3. Konfiguráció C MQTT TLS 1.3 felett, kölcsönös tanúsítványalapú hitelesítéssel (mTLS) és ACL-ekkel

- Kommunikáció: megegyezik a B konfigurációval (MQTT over TLS 1.3).
- Hitelesítés: a broker csak olyan klienseket fogad el, amelyek érvényes, a saját CA által aláírt kliens-tanúsítvánnyal rendelkeznek (mTLS).
- Hozzáférés-szabályozás: brokeroldali ACL-ek határozzák meg, hogy az egyes tanúsítvány-azonosítók mely topicokra publikálhatnak, illetve melyekre iratkozhatnak fel.
- Cél: annak demonstrálása, hogy a Zero Trust szemléletnek megfelelően a csatornaszintű védelem és az erős eszközidentitás kombinációja hogyan szűkíti le a támadási felületet.

A három konfiguráció között a különbség kizárólag a broker és a hitelesítés beállításáiban jelenik meg; a szimulált szenzorok kódja, a mintavételezési periódus és a backend működése változatlan marad, így az eredmények közvetlenül összehasonlíthatók.

5.3.2 Mérőszámok és kiértékelési szempontok

A vizsgálat során az alábbi fő mérőszámok kerülnek meghatározásra:

- **Üzenetszám** **szenzoronként** **(N_msg)**
Az adott mérési futás során beérkezett üzenetek darabszáma szenzoronként. Ez jelzi, hogy történt-e üzenetvesztés vagy a biztonsági beállítások okoztak-e sorozatos kapcsolódási problémákat.
- **Mérés** **időtartama** **(T_run)**
A collector által rögzített első és utolsó üzenet fogadási ideje közötti különbség. Ebből a mutatóból, az ismert mintavételezési periódussal együtt következtetni lehet arra, hogy mennyire stabil a kommunikáció.
- **Üzenetküldési** **ráta** **(λ)**
A fenti két értékből számolva:

$$\lambda = \frac{N_msg}{T_run}$$

- amely megmutatja, hogy az adott szenzor átlagosan hány üzenetet tudott másodpercenként a broker felé továbbítani. A baseline és a TLS/mTLS konfigurációk közötti különbség jól jelzi a kriptográfiai réteg overheadjét.
- **Átlagos mért hőmérséklet ($\bar{T}_{\text{sensor}}$)**
Bár önmagában nem biztonsági mutató, az átlagos hőmérséklet ellenőrzésére szolgál egyrészt, mint „egészségügyi” ellenőrzés (nem torzult-e el a minta), másrészt a támadási scenáriók során (pl. jogosulatlan üzenetbefecskendezés) jól látható, ha egy szenzor topicján irreális értékek jelennek meg.
- **Elutasított kapcsolatok és jogosulatlan kísérletek száma (N_{denied})**
A C konfigurációban külön figyelmet kap, hogy a broker logjaiban hány olyan próbálkozás jelenik meg, ahol:
 - a kliens nem rendelkezik érvényes tanúsítvánnyal, vagy
 - érvényes tanúsítvánnyal, de ACL által tiltott topicra próbál publikálni.

A fenti értékeket az `analyze_measurements.py` és annak TLS/mTLS konfigurációhoz igazított változatai számítják ki a `measurements*.csv` állományok alapján.

5.3.3 Mérési eljárás

A mérések minden konfigurációban egységes, reprodukálható eljárás szerint zajlanak:

1. Környezet inicializálása

- Elindul a megfelelő Mosquitto-konfiguráció (A: titkosítatlan, B–C: TLS 1.3, C esetén mTLS + ACL).
- A Python collector program (`collector.py`, `collector_tls.py` vagy mTLS-es változat) csatlakozik a brokerhez és feliratkozik az `iot/lab/#` topic-családra.

2. Szenzorok szimulációja

- A Wokwi-ban futó `sensor1` és `sensor2` projektek csatlakoznak a brokerhez.
- Mindkét szenzor 5 másodperces periódussal publikál hőmérsékleti értékeket a saját topicjára. A payload tartalmazza a szenzorazonosítót és az eszköz oldali időbélyeget is.

3. Mérési időablak

- Egy mérési futás hossza jellemzően **1–2 perc**, ami elegendő adatpontot szolgáltat az üzenetküldési ráta és az átlaghőmérséklet stabil becsléséhez.

- Ugyanazt az időablakot használjuk a három konfiguráció esetén, így a különbségek nem a futás hosszából adódnak.

4. Adatgyűjtés és -mentés

- A collector program valós időben rögzíti a mért adatokat a measurements*.csv fájlokba.
- A mérés végén a Wokwi-szimuláció leáll, a collector kilép, a CSV-fájl lezárul.

5. Kiértékelés

- Az elemző szkript feldolgozza az adott konfigurációhoz tartozó CSV-fájlt, kiszámítja a fenti mérőszámokat, és a konzolra írja azokat.
- Az eredmények táblázatos formában bekerülnek az 5. fejezet vonatkozó alfejezeteibe, illetve grafikonok formájában az ábrák közé.

Ezzel a módszerrel a három konfiguráció egymástól csak biztonsági szintben tér el; minden más paraméter (szenzorok száma, mintavételezési periódus, payload-struktúra) azonos.

5.3.4 Támadási scenáriók

A mérések nem kizárólag „normál” üzemet vizsgálnak, hanem olyan kontrollált támadási helyzeteket is, amelyek jól illusztrálják az egyes védelmi mechanizmusok szerepét. Ezek közül a legfontosabbak:

1. Jogosulatlan publish titkosítatlan brokerre (Konfiguráció A)

- Egy külön Python „támadó kliens” a szenzorokéhoz hasonló formátumú üzeneteket küld a `iot/lab/sensor1/temperature` topicra.
- Mivel sem TLS, sem hitelesítés nincs, a broker gond nélkül elfogadja az üzeneteket; a collector szemszögéből a „valódi” és a hamis szenzoradatok összekeverednek.
- Az átlaghőmérséklet és a mért értékek eloszlása alapján jól kimutatható az adatinjekció hatása.

2. Jogosulatlan publish TLS mellett, de hitelesítés nélkül (Konfiguráció B)

- A támadó kliens TLS-sel csatlakozik a brokerhez, de továbbra sem használ erős eszközidentitást.
- A broker a titkosított csatorna ellenére nem tud különbséget tenni a jogos és jogosulatlan kliensek között, így az adatinjekció ugyanúgy sikeres.
- Ez demonstrálja, hogy a TLS önmagában nem elég: a csatorna védett, de a végpontok identitása gyenge.

3. Jogosulatlan publish mTLS + ACL mellett (Konfiguráció C)

- A támadó kliens vagy:
 - nem rendelkezik érvényes kliens-tanúsítvánnyal → a broker elutasítja a TLS handshake-et,
 - vagy érvényes tanúsítvánnyal, de ACL által tiltott topicra próbál publikálni → a broker not authorized hibát ad.
- A collector CSV-fájljában nem jelennek meg a támadó által küldött üzenetek; a broker logjában viszont rögzülnek az elutasított kísérletek.

A három scenárió egymásra épül: míg az A és B konfigurációban a támadás sikeres, addig a C konfigurációban a mTLS és az ACL-ek kombinációja megakadályozza, hogy a támadó érdemben befolyásolja a szenzoradatokat. Ez közvetlenül illusztrálja a 4.3 fejezetben tárgyalt Zero Trust és defense-in-depth elvek gyakorlati hasznát.

5.4 Mérési eredmények és értékelés

Ebben az alfejezetben a 5.1–5.3 fejezetekben bemutatott szimulációs környezetben végzett mérések eredményeit foglalom össze. A cél annak vizsgálata, hogy a különböző biztonsági beállítások (titkosítatlan MQTT, TLS 1.3 feletti MQTT) milyen hatással vannak az üzenetküldési rátára és a rendszer viselkedésére, illetve milyen mértékben növelik a kommunikáció biztonságát.

A mérések során minden konfigurációra ugyanazt a módszertant alkalmaztam:

- két ESP32-alapú szenzor (sensor1, sensor2) 5 másodperces periódussal publikált hőmérsékleti értékeket,
- a collector modul a measurements*.csv fájlokba rögzítette a beérkező üzeneteket,
- az analyzer szkript szenzoronként kiszámította az üzenetszámot, a futás időtartamát, az üzenetküldési rátát és az átlagos hőmérsékletet.

5.4.1 Baseline – titkosítatlan MQTT (Konfiguráció A)

A baseline mérések során a szenzorok egy titkosítatlan, hitelesítés nélküli brokerre csatlakoztak (TCP/1883). A collector program a measurements.csv állományba írt minden beérkező üzenetet, az analyzer pedig szenzoronként aggregálta az adatokat.

Az 5.4.1. táblázat egy tipikus futás eredményeit mutatja be (a konkrét értékek a measurements.csv aktuális tartalmából származnak):

V.I. táblázat – Baseline (titkosítatlan) mérési eredmények egy tipikus futásban

Szenzor	Üzenetszám N_{msg}	Időtartam T_{run} [s]	Ráta λ [msg/s]	Átlagos hőmérséklet $\bar{T}$ [°C]
sensor1	60	905.4	0.066	24.95
sensor2	60	915.8	0.066	24.85

(forrás: saját mérés)

A baseline futás során mindkét szenzor esetében 60 üzenet érkezett, körülbelül 905–916 másodperces időablakban. Ez üzenetküldési rátának felel meg, vagyis egy üzenet átlagosan nagyjából 15 másodpercenként jut el a brokerhez. A névlegesen 5 másodperces mintavételi periódus tehát a gyakorlatban jelentősen torzul, ami elsősorban a Wokwi szimulációs környezet, illetve a publikus, interneten keresztül elérhető test.mosquitto.org broker változó hálózati késleltetéséből és terheléséből adódik.

A mérések során ugyanakkor nem jelentkezett tartós üzenetvesztés: mindkét szenzor esetében a collector a várt nagyságrendnek megfelelő, 60 darab körüli N_{msg} értéket rögzített. A hőmérsékleti értékek (a valóságban véletlenszámok) átlagai a 20–30 °C közötti tartományban maradtak (sensor1: 24,95 °C, sensor2: 24,85 °C), ami azt jelzi, hogy a payloadok formailag épek, a JSON-parsing nem eredményezett torzulást.

Biztonsági szempontból a baseline konfiguráció kifejezetten gyenge:

- a forgalom titkosítatlanul halad át a hálózaton,
- a broker nem azonosítja a klienseket,
- bármely külső kliens szabadon küldhet üzeneteket a szenzorok topicjaira, illetve le is hallgathatja azokat.

Ezt a sebezhetőséget a 5.3.4. alfejezetben bemutatott „jogosulatlan publish” szcenárió jól illusztrálja: a támadó kliens által küldött hamis hőmérsékleti értékek a collector számára megkülönböztethetetlenek a valódi szenzoradatokról, így a feldolgozott statisztikák is torzulnak. Ez indokolja a továbblépést a TLS-es és mTLS + ACL konfigurációk irányába.

5.4.2 TLS 1.3 feletti MQTT szerverhitelesítés (Konfiguráció B)

A második mérési konfigurációban a szenzorok és a collector már a TLS-es Mosquitto brokerhez csatlakoztak (TCP/8883). Ebben az esetben a mérések lokális brokerrel futottak, ezért a 5.4.2. táblázatban szereplő üzenetküldési ráták magasabbak, mint a publikus test.mosquitto.org brokerrel végzett baseline futásban – az abszolút értékek így nem

közvetlenül, hanem csak nagyságrendileg hasonlíthatók össze. A broker egy saját CA által aláírt szervertanúsítvánnyal azonosítja magát, a kliensek pedig a CA-tanúsítvány (ca.crt) segítségével ellenőrzik a szerver identitását. A collector_tls.py modul a measurements_tls.csv állományba rögzíti az adatokat, az analyzer módosított változata pedig ugyanazokat a mutatókat számítja ki, mint a baseline esetben. Az 5.4.2. táblázat egy tipikus TLS-es futás eredményeit foglalja össze:

V.II. táblázat TLS-es MQTT mérési eredmények egy tipikus futásban

Szenzor	Üzenetszám N_msg	Időtartam T_run [s]	Ráta λ [msg/s]	Átlagos hőmérséklet $\bar{T}$ [°C]
sensor1	60	295.1	0.203	25.18
sensor2	60	295.5	0.203	24.68

(forrás: saját mérés)

A TLS-es futások legfontosabb megfigyelései a V.II. táblázat alapján a következők.

A vizsgált futásban mindkét szenzor esetén 60 üzenet érkezett nagyjából 295 másodperces időablakban, ami körülbelül $\lambda \approx 0,203$ msg/s üzenetküldési rátának felel meg. Ez azt jelenti, hogy egy üzenet átlagosan közel 5 másodpercenként jutott el a brokerhez, vagyis a gyakorlatban jól közelíti a szenzorkódban beállított mintavételi periódust. A mérési eredmények alapján a TLS 1.3 bevezetése ebben a lokális, kisméretű szimulációs környezetben nem okozott érzékelhető teljesítménycsökkenést.

A collector_tls.py ugyanazt a JSON payload-struktúrát és CSV-formátumot használja, mint a titkosítatlan collector. Ennek megfelelően az üzenetszámok és az átlaghőmérsékletek a baseline mérésekhez hasonló tartományban maradnak. A mért átlaghőmérsékletek sensor1 esetében 25,18 °C, sensor2 esetében 24,68 °C voltak. Ez alátámasztja, hogy a TLS réteg beiktatása a rendszer funkcionalitását nem módosította, csupán a csatorna biztonságát növelte.

Biztonsági szempontból ugyanakkor lényeges előrelépés történt: a forgalom titkosított, a szenzoradatok út közben nem olvashatók és nem módosíthatók egyszerűen, és a kliensek csak akkor fogadják el a kapcsolatot, ha a broker tanúsítványa a megbízható CA-tól származik. A TLS tehát érdemi védelmet biztosít a lehallgatás és a man-in-the-middle típusú támadások ellen.

Ugyanakkor a TLS önmagában nem oldja meg a jogosulatlan kliensek problémáját: ha egy támadó kliens ismeri a broker címét és rendelkezik a CA-tanúsítvánnyal, ő is létre tud hozni egy titkosított kapcsolatot, és publikálhat hamis adatokat olyan topicokra, amelyekhez nincs explicit hozzáférés-szabályozás rendelve. Ez indokolja a következő lépcsőt, a mTLS és az ACL-ek bevezetését.

5.4.3 Összehasonlító értékelés

A baseline (Konfiguráció A) és a TLS-es (Konfiguráció B) futások eredményei alapján több szempont szerint is összehasonlítható a rendszer viselkedése.

- **Teljesítmény szempontjából**

A baseline futásban mindkét szenzor esetében 60 üzenet érkezett nagyjából 905–916 másodperc alatt, ami üzenetküldési rátának felel meg. A TLS-es konfigurációban ugyancsak 60–60 üzenet érkezett, viszont egy ~295 másodperces időablakban, ami rátát eredményez. Az abszolút értékek közvetlen összehasonlítása során figyelembe kell venni, hogy a baseline esetben a szenzorok egy publikus, interneten keresztül elérhető brokerhez (test.mosquitto.org) kapcsolódnak, míg a TLS-es futás lokális Mosquitto brokerrel történt. A λ_A és λ_B közötti különbség tehát elsősorban a hálózati környezet (internet-latencia, publikus broker terhelése) eltéréseiből adódik, és nem a TLS protokoll többletköltségének közvetlen következménye.

A lokális TLS-es futás azt mutatja, hogy két szenzor és egy collector esetén, ~5 másodperces publikálási periódussal a rendszer gond nélkül képes a terhelés kiszolgálására, a CPU- és memóriaigény nem válik szűk keresztmetszetté. Ez összhangban áll a szakirodalommal, amely szerint a TLS 1.3 bevezetése elsősorban a kapcsolatfelépítésnél jelent fix költséget, míg tartós kapcsolatoknál az egyes üzenetekre jutó overhead viszonylag kicsi, különösen ilyen alacsony IoT-mintavételezési ráta mellett.

- **Biztonsági szempontból**

A baseline konfigurációban a rendszer gyakorlatilag egy „nyitott MQTT buszként” viselkedik: az adatforgalom titkosítatlan, a broker nem hitelesíti a klienseket, és a topicokra vonatkozó hozzáférés-szabályozás sincs érvényben. A 5.3.4. alfejezetben bemutatott támadási szcenárió azt szemlélteti, hogy egy jogosulatlan kliens is képes a szenzorok topicjaira hamis adatokat publikálni, amelyek a collector és az elemző modul számára megkülönböztethetetlenek a legitim üzenetektől. Ez az üzleti logika szintjén a döntéstámogató statisztikák torzulásához vezethet.

A TLS-es konfiguráció első lépésben a kommunikációs csatorna bizalmasságát és integritását erősíti: a szenzoradatok út közben nem olvashatók és nem módosíthatók egyszerűen, és a kliensek csak akkor fogadják el a kapcsolatot, ha a broker tanúsítványa a megbízható CA-tól származik. Ezzel kizárható a passzív lehallgatási és a klasszikus man-in-the-middle típusú támadások. Ugyanakkor a TLS önmagában még nem akadályozza meg, hogy egy támadó kliens (amely ismeri a broker címét és rendelkezik a CA-tanúsítvánnyal) titkosított csatornán csatlakozzon és jogosulatlan topikokra publikáljon. A jogosulatlan kliensek teljes kizárásához szükséges az erős kliensidentitás (mTLS) és a finomhangolt hozzáférés-vezérlési listák (ACL-ek) bevezetése, ahogy azt a Konfiguráció C-ben vázolt architektúra bemutatja.

- **Kutatási kérdések szempontjából**

A mérések a dolgozatban megfogalmazott hipotéziseket részben megerősítik. Egyrészt a baseline futás rávilágít arra, hogy egy egyszerű, autentikáció nélküli MQTT-architektúra milyen könnyen támadható: a jogosulatlan publish kísérletek detektálása és blokkolása önmagában a broker szintjén nem történik meg. Másrészt a TLS-es konfiguráció demonstrálja, hogy egy IoT-környezetre jellemző, alacsony mintavételezési ráta mellett a TLS 1.3 bevezetése nem jár drasztikus teljesítményromlással, miközben érdemben erősíti a kommunikációs csatorna biztonságát. A dolgozat következő fejezete (Konfiguráció C) olyan mTLS + ACL alapú megoldást vázol fel, amely a csatornaszintű titkosításra építve már a jogosulatlan kliensek és publish kísérletek blokkolását is célozza, és ezzel teljesebb választ ad a bevezetésben megfogalmazott kutatási kérdésekre.

6. A fejlesztett (szimulált) IoT-biztonsági rendszer bemutatása

6.1 Architektúra logikai felépítése

A dolgozat gyakorlati részében kialakított szimulációs környezet célja egy olyan **IoT-biztonsági tesztlabor** megvalósítása, amelyben jól kontrollálható feltételek mellett vizsgálhatók az MQTT-alapú kommunikáció sebezhetőségei és a különböző védelmi mechanizmusok hatásai. Az architektúra központi eleme egy Mosquitto MQTT-broker, amelyhez szimulált ESP32 szenzorok, mérési backend komponensek és – bizonyos szcenáriókban – támadó kliensek csatlakoznak.

A rendszer logikai felépítése négy fő rétegre bontható:

1. **Periféria réteg – szimulált IoT-eszközök (sensor1, sensor2)**

A Wokwi környezetben futó két ESP32-alapú szenzor projekt (sensor1 és sensor2) reprezentálja az IoT-eszközöket. Mindkét node periodikusan hőmérsékleti mérést szimulál, és az adatokat MQTT üzenet formájában publikálja a

iot/lab/sensorX/temperature topicokra. A szenzorok kizárólag publish műveletet végeznek, subscribe jogosultsággal nem rendelkeznek.

2. Kommunikációs réteg – MQTT-broker

A Mosquitto broker tölti be a központi üzenetközvetítő szerepét. Három, logikailag elkülönülő konfigurációt különböztetnek meg:

- **Konfiguráció A – baseline (plaintext MQTT, 1883):** titkosítás és hitelesítés nélküli környezet, amely a legrosszabb gyakorlatot demonstrálja, és jól szemlélteti a támadási felület nagyságát.
- **Konfiguráció B – TLS 1.3 feletti MQTT (8883):** a kommunikáció TLS-sel védett, a broker szervertanúsítvánnyal igazolja magát, a kliensek pedig egy saját CA (ca.crt) alapján ellenőrzik a szerver identitását.
- **Konfiguráció C – mTLS + ACL (koncepcionális):** a későbbi bővítésre szánt konfiguráció, ahol a kliensek egyedi tanúsítványokkal azonosítják magukat (mutual TLS), és a broker topic-szintű hozzáférés-szabályozási listák (ACL) alapján dönt a publish/subscribe műveletek engedélyezéséről.

A három konfiguráció ugyanarra a logikai topológiára épül, de eltérő biztonsági szintet képvisel, ez teszi lehetővé az 5. fejezetben bemutatott összehasonlító méréseket.

3. Backend réteg – adatgyűjtés és elemzés

A Python-alapú backend két fő komponensből áll:

- **collector.py / collector_tls.py:** MQTT-kliensként feliratkoznak a *iot/lab/sensor+/temperature* topicokra, majd a beérkező üzeneteket időbélyeggel együtt a *measurements.csv* illetve *measurements_tls.csv* állományokba rögzítik. A TLS-es változat a ca.crt segítségével ellenőrzi a broker tanúsítványát.
- **analyze_measurements.py / analyze_measurements_tls.py:** feldolgozzák a mért adatokat, szenzoronként kiszámítják az üzenetszámot, a futási időt, az üzenetküldési rátát és az átlaghőmérsékletet. Ezek a mutatók képezik az 5.4. fejezetben bemutatott táblázatok alapját.

A backend a gyakorlatban egy „megfigyelő” komponensként működik: nem avatkozik be az MQTT-forgalomba, kizárólag passzívan rögzíti és elemzi azt. Ez összhangban van azzal a céllal, hogy a vizsgálat a különböző brokerkonfigurációk hatását mérje, ne pedig egy aktív védelmi modul működését.

4. Támadói réteg – jogosulatlan kliensek

A 7. fejezetben részletezett támadási scénáriókhoz egy vagy több további MQTT-kliens

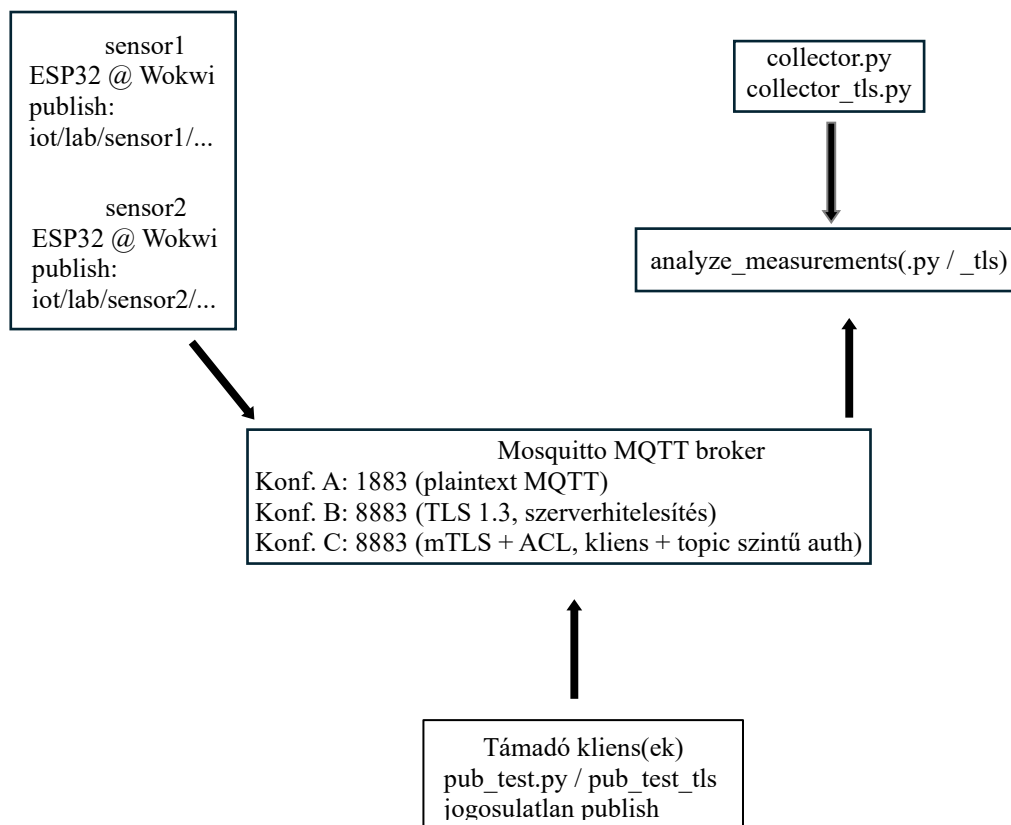
tartozik, amelyek ugyanarra a brokerre csatlakoznak, mint a szenzorok és a backend.

Baseline és TLS konfigurációban a támadó kliens képes:

- feliratkozni a *iot/lab/sensor1/temperature* topicra,
 - saját, hamis hőmérsékleti értékeket publish-elni ugyanebbe a topicba, ezáltal összekeverve a valódi és a manipulált adatokat a collector CSV-jében.
- A koncepcionális mTLS + ACL konfigurációban ezzel szemben a támadó kliens – tanúsítvány hiányában vagy megfelelő ACL jogosultság nélkül – elvileg elutasításra kerül, amit a broker naplói és a mért N_{denied} értékek jeleznek.

Az architektúra egészét egy **többrétegű védelmi modell** fogja össze. A szenzorok szintjén a minimális jogosultság elve érvényesül (csak publish, szigorúan egy topicra), a kommunikációs rétegen TLS alapú csatornavédelem és – a tervezett konfigurációban – mTLS alapú eszközidentitás jelenik meg, míg a backend réteg felel a részletes naplózásért és az adatok későbbi elemzéséért. A 4. fejezetben ismertetett Zero Trust elvek így konkrét, implementált komponensek formájában jelennek meg: a broker nem feltételez implicit bizalmat egyetlen klienssel szemben sem, a hozzáférés minden esetben explicit policy-k és, a későbbi bővítés során, tanúsítvány-alapú identitás alapján történik.

6.1. ábra – A szimulált IoT-biztonsági rendszer logikai architektúrája



Forrás: saját szerkesztés

6.2 ESP32-alapú szenzor node-ok implementációja

A szimulált IoT-rendszer peremén két darab ESP32-alapú szenzor node helyezkedik el (sensor1 és sensor2), amelyek a Wokwi online környezetében futnak. A node-ok Arduino-stílusú C/C++ nyelven írt firmware-t futtatnak, és mindketten MQTT kliensként viselkednek: periódikusan hőmérsékleti értéket generálnak, majd JSON formátumú payloadot publikálnak a saját topicjukra. A két eszköz implementációja szándékosan minimális mértékben tér el egymástól, így a különbségek a mérésekben elsősorban a hálózati és brokeroldali tényezőknek tudhatók be.

6.2.1 WiFi-kapcsolat inicializálása Wokwi környezetben

A Wokwi szimulátor egy virtuális WiFi-hálózatot biztosít „*Wokwi-GUEST*” SSID-n, jelszó nélkül. Ennek megfelelően a firmware-ben a következő paraméterek szerepelnek:

```
const char* ssid    = "Wokwi-GUEST";  
const char* password = "";
```

Forrás: saját szerkesztés

A `setupWifi()` függvény hívja meg a csatlakozást:

```
void setupWifi() {  
    Serial.print("Connecting to WiFi");  
    WiFi.begin(ssid, password);  
  
    while (WiFi.status() != WL_CONNECTED) {  
        delay(500);  
        Serial.print(".");  
    }  
  
    Serial.println();  
    Serial.print("WiFi connected, IP address: ");  
    Serial.println(WiFi.localIP());  
}
```

Forrás: saját szerkesztés

A fenti megoldás egyrészt biztosítja, hogy a node csak sikeres WiFi-kapcsolat után lép tovább az MQTT-réteg inicializálására, másrészt a soros kimeneten is jól követhetővé teszi a csatlakozási folyamatot, ami a Wokwi konzoljában újít hasznos visszajelzést.

6.2.2 MQTT-kliens inicializálása és kapcsolatfelépítés

Az MQTT-kommunikációhoz a firmware a PubSubClient könyvtárat használja. A dolgozatban bemutatott baseline mérések esetén a Wokwi-ból közvetlenül egy publikus MQTT-brokerre (test.mosquitto.org) történik a csatlakozás, titkosítás nélkül:

```
#include <PubSubClient.h>

const char* mqtt_server = "test.mosquitto.org";
const int  mqtt_port  = 1883;

WiFiClient espClient;
PubSubClient client(espClient);
```

Forrás: saját szerkesztés

A kliens beállítása a setup() függvény végén történik:

```
void setup() {
  Serial.begin(115200);
  delay(1000);

  randomSeed(analogRead(0));    // véletlenszám-generálás inicializálása

  setupWifi();                  // WiFi-kapcsolat felépítése
  client.setServer(mqtt_server, mqtt_port); // MQTT broker címe
}
```

Forrás: saját szerkesztés

A loop() függvény elején minden ciklusban ellenőrzésre kerül, hogy az MQTT-kapcsolat él-e; ha nem, a reconnect() függvény próbál újra csatlakozni:

```
void reconnect() {
  while (!client.connected()) {
    Serial.print("Attempting MQTT connection...");
    String clientId = "esp32-sensor1-"; // sensor2-nél "esp32-sensor2-"
    clientId += String(random(0xffff), HEX);

    if (client.connect(clientId.c_str())) {
      Serial.println("connected");
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      delay(5000);
    }
  }
}
```

Forrás: saját szerkesztés

A fenti minta biztosítja, hogy egy átmeneti broker- vagy hálózati hiba esetén a node automatikusan megpróbálja visszaállítani az MQTT-kapcsolatot, ami az IoT-szenzorok robusztus működése szempontjából fontos követelmény.

6.2.3 JSON payload és topic-struktúra

Mindkét szenzor azonos, hierarchikus MQTT-topic-struktúrát használ:

- iot/lab/sensor1/temperature
- iot/lab/sensor2/temperature

A payload a 5.1.3 alfejezetben is bemutatott, egységes JSON-formátumot követi:

```
{
  "sensor_id": "sensor1",
  "ts_device": 12345,
  "temperature": 25
}
```

Forrás: saját szerkesztés

A JSON-üzenet felépítése a firmware-ben a következőképpen történik:

```
unsigned long now = millis();
int temperature = random(20, 30); // 20–29 °C közötti random érték

char payload[128];
snprintf(payload, sizeof(payload),
    "{\"sensor_id\":\"sensor1\",\"ts_device\":%lu,\"temperature\":%d}\",",
    now, temperature);

const char* topic = "iot/lab/sensor1/temperature";
client.publish(topic, payload);
```

Forrás: saját szerkesztés

A sensor2 implementációja annyiban tér el, hogy a sensor_id mező értéke "sensor2", és a topic az iot/lab/sensor2/temperature. Ez a minimális különbség biztosítja, hogy a backend oldalon (collector és analyzer modulok) szenzorazonosító alapján külön-külön lehessen aggregálni és értelmezni az adatokat.

6.2.4 Időzítés és adatszórás: 5 másodperces periódus

A szenzorok mintavételezési periódusát a loop() függvényben egy egyszerű időzítés valósítja meg a millis() függvény segítségével. A cél, hogy névlegesen 5 másodpercenként kerüljön sor egy-egy mérésre és publish műveletre:

```
unsigned long lastMsg = 0;

void loop() {
    if(!client.connected()) {
        reconnect();
    }
    client.loop();

    unsigned long now = millis();
    if (now - lastMsg > 5000) { // 5 másodpercenként küldünk egy mérést
        lastMsg = now;

        int temperature = random(20, 30);
        // JSON payload összeállítása és publish (ld. előző alfejezet)
    }
}
```

Forrás: saját szerkesztés

A random hőmérsékleti érték generálása (`random(20, 30)`) szándékosan egyszerű; célja nem a valós fizikai modell leképezése, hanem a teljes adatút (`payload → MQTT → broker → collector → analyzer`) tesztelése változó, de jól kontrollálható mintákon. A `randomSeed(analogRead(0))` hívás gondoskodik arról, hogy a szimuláció induláskor a véletlenszám-generátor állapota pszeudorandom legyen, így a mért hőmérsékleti görbék futásonként eltérnek.

A 5.4.1 alfejezetben bemutatott baseline eredmények rámutatnak arra, hogy a névlegesen 5 másodperces periódus a gyakorlatban a Wokwi és a publikus MQTT-broker (`test.mosquitto.org`) miatt körülbelül 15 másodperces effektív üzenetközi időre torzul. A firmware szintjén ugyanakkor a fenti időzítési logika implementálása biztosítja, hogy a szenzoroldali kód determinisztikus és jól reprodukálható legyen; a további késleltetések már a hálózati és brokeroldali tényezőkből adódnak.

6.2.5 sensor1 és sensor2 szerepe a mérésekben

A két szenzor implementációja gyakorlatilag azonos, mindössze az alábbi paraméterekben térnek el:

- az MQTT `clientId` előtagja ("`esp32-sensor1-`" vs. "`esp32-sensor2-`"),
- a `sensor_id` mező értéke a JSON payloadban ("`sensor1`" vs. "`sensor2`"),
- a publikálásra használt topic (`iot/lab/sensor1/temperature` vs. `iot/lab/sensor2/temperature`).

Ez a minimális különbség két fontos célt szolgál:

1. **Összehasonlíthatóság:** mivel a szenzorlogika és a mintavételezési periódus azonos, a 5.4. fejezetben bemutatott eredmények összevethetők szenzoronként; a különbségek elsősorban a hálózati fluktuációból erednek, nem a kódbeli eltérésekből.
2. **Skálázhatóság:** a jelenlegi két node könnyen kiterjeszthető nagyobb szenzorparkra is, amelyben további ESP32-esek csatlakoznának új topicokra (pl. `iot/lab/sensor3/temperature`, `iot/lab/sensor4/humidity` stb.). Az implementáció sablonja ezt a fajta bővítést jól támogatja.

Összességében elmondható, hogy az ESP32-alapú szenzor node-ok implementációja szándékosan egyszerű és transzparens: a kód jól érthető módon valósítja meg a WiFi-kapcsolat felépítését, az MQTT-kliens működését, a JSON payload előállítását és a periodikus adatszórását. Ez nemcsak a mérések reprodukálhatóságát segíti, hanem a későbbi bővítések (pl. valós szenzorok integrálása, batteriakezelés, alvó módok) számára is stabil kiindulópontot jelent.

6.3 MQTT-broker és Mosquitto-konfigurációk

A szimulációs környezet központi eleme az Eclipse Mosquitto MQTT-broker, amely a különböző biztonsági beállítások szerint három fő konfigurációban került kialakításra. Ezek a 5.3. alfejezetben bemutatott Konfiguráció A (plaintext), Konfiguráció B (TLS 1.3 feletti MQTT) és Konfiguráció C (mTLS + ACL koncepcionális bővítés). Ebben az alfejezetben a konfigurációk gyakorlati, Mosquitto-szintű megvalósítását foglalom össze.

6.3.1 Konfiguráció A – titkosítatlan MQTT (1883)

A baseline mérési környezetben a broker a klasszikus, titkosítatlan MQTT-forgalmat fogadja a 1883-as TCP porton. Ez a beállítás felel meg annak a „rossz gyakorlatnak”, amelyet a dolgozat kiindulópontként vizsgál, és amely egyben a legnagyobb támadási felületet biztosítja.

A Mosquitto alapértelmezett konfigurációja Windows alatt már tartalmaz egy egyszerű beállítást az 1883-as listenerre, a dolgozatban használt minimális konfiguráció ennek megfelelően:

```
listener 1883
protocol mqtt

allow_anonymous true
```

Forrás: saját szerkesztés

A listener 1883 direktíva azt jelzi, hogy a broker a 1883-as TCP porton fogadja az MQTT-kapcsolatokat, a protocol mqtt pedig a klasszikus MQTT 3.1.1/5.0 protokoll használatát írja elő. Az allow_anonymous true opció engedélyezi, hogy a kliensek felhasználónév/jelszó megadása nélkül csatlakozzanak – ez funkcionális szempontból leegyszerűsíti a szimulációt, ugyanakkor biztonságilag kifejezetten gyenge beállítás.

Ebben a konfigurációban csatlakozik a Wokwi-ban futó *sensor1* és *sensor2* (a test.mosquitto.org publikus brokerre mutató beállításaiikkal), illetve a collector.py modul is, amely feliratkozik az *iot/lab/#* topicokra, és a 5.4.1. táblázatban bemutatott baseline mérési eredményeket rögzíti.

6.3.2 Konfiguráció B – TLS 1.3 feletti MQTT (8883)

A második konfiguráció célja a kommunikációs csatorna titkosítása, valamint a broker szerveridentitásának hitelesítése TLS 1.3 használatával. Ennek érdekében a Mosquitto külön, TLS-képes konfigurációs állománnyal indul (pl. mosquitto_tls.conf), amely a következő kulcsparamétereket tartalmazza:

```
listener 8883
protocol mqtt

cafile "C:/Program Files/mosquitto/certs/ca.crt"
certfile "C:/Program Files/mosquitto/certs/server.crt"
keyfile "C:/Program Files/mosquitto/certs/server.key"

allow_anonymous true
```

Forrás: saját szerkesztés

A listener 8883 itt a TLS-sel védett MQTT („MQTTS”) alapértelmezett portját jelöli. A cafile a saját tanúsítványkiadó (CA) nyilvános tanúsítványát (ca.crt), a certfile a broker szervertanúsítványát (server.crt), a keyfile pedig a hozzá tartozó privát kulcsot (server.key) adja meg. Ezeket a tanúsítványokat és kulcsfájlokat a dolgozatban egy egyszerű, ön aláírt CA és a hozzá tartozó szervertanúsítvány generálása biztosítja.

A mérési futásokban a collector_tls.py és a TLS-es tesztkliens-szkriptek (pl. pub_test_tls.py) ezen a 8883-as listeneren keresztül csatlakoznak, és a ca.crt segítségével ellenőrzik, hogy a broker valóban a várt CA által aláírt tanúsítvánnyal rendelkezik. A kliensoldali beállítás ennek megfelelően:

- a paho-mqtt kliens TLS-módba állítása (tls_set(ca_certs="ca.crt", ...)),
- a broker címének localhost-ra állítása,
- a 8883-as port használata.

Ebben a konfigurációban a broker továbbra is engedélyezi az anonim hozzáférést (allow_anonymous true), tehát a kliensek tanúsítvány vagy felhasználónév/jelszó nélkül csatlakozhatnak. A TLS bevezetése így elsősorban a csatorna bizalmasságát és integritását, valamint a szerverhitelesítést javítja, de a jogosulatlan kliensek kérdését önmagában még nem kezeli.

A 5.4.2. táblázatban bemutatott TLS-es mérési eredmények ebben a konfigurációban születtek: a collector_tls.py a titkosított csatornán érkező üzeneteket rögzíti a measurements_tls.csv fájlba, amelyet az analyze_measurements_tls.py dolgoz fel.

6.3.3 Konfiguráció C – mTLS + ACL (konceptcionális bővítés)

A dolgozat harmadik, koncepcionális konfigurációja (Konfiguráció C) a Zero Trust elveknek megfelelően nemcsak a csatornaszintű titkosítást és szerverhitelesítést célozza, hanem a kliensek erős azonosítását és a topic-szintű hozzáférés-szabályozást is. Ezt a gyakorlatban a Mosquitto két kiegészítő mechanizmussal tudja támogatni:

- **kölcsönös TLS (mTLS):** a kliensek is tanúsítvánnyal igazolják magukat a broker felé;
- **ACL-ek (Access Control List):** a broker felhasználónév (vagy tanúsítványból származtatott identity) alapján dönti el, hogy egy kliens mely topicokra publikálhat és mely topicokra iratkozhat fel.

Egy tipikus mTLS + ACL konfiguráció a következő elemekből állna:

```
listener 8883
protocol mqtt

cafile "C:/Program Files/mosquitto/certs/ca.crt"
certfile "C:/Program Files/mosquitto/certs/server.crt"
keyfile "C:/Program Files/mosquitto/certs/server.key"

require_certificate true
use_identity_as_username true

allow_anonymous false
password_file "C:/Program Files/mosquitto/passwd"
acl_file "C:/Program Files/mosquitto/acl.conf"
```

Forrás: saját szerkesztés

A `require_certificate true` előírja, hogy a kliensek csak akkor csatlakozhatnak, ha érvényes kliens-tanúsítvánnyal rendelkeznek, amelyet a megadott CA írt alá. A `use_identity_as_username true` beállítás lehetővé teszi, hogy a broker a kliens tanúsítványában szereplő CN (Common Name) vagy más azonosító mező alapján azonosítsa a klienst, és ezt használja felhasználónévként az ACL-ellenőrzés során.

Az `allow_anonymous false` kikapcsolja az anonim hozzáférést; a `password_file` és `acl_file` pedig a jelszó- és ACL-adatbázis helyét definiálja. Az ACL-állományban például a következő szabályok szerepelhetnének:


```
user sensor1
topic write iot/lab/sensor1/temperature

user sensor2
topic write iot/lab/sensor2/temperature

user collector
topic read iot/lab/#

user attacker
topic read iot/lab/#
topic write iot/lab/testsensor/#
```

Forrás: saját szerkesztés

A fenti példa jól illusztrálja, hogyan válik megvalósíthatóvá a minimális jogosultság elve: a *sensor1* csak a saját hőmérséklet-topicjára publikálhat, a *sensor2* a sajátjára, a collector csak olvas (subscribe) jogosultsággal rendelkezik az *iot/lab/#* topiccsaládra, míg az esetleges „támadó” kliens is legfeljebb egy izolált, teszt célú topicba írhat. Egy olyan kliens, amely nem rendelkezik megfelelő tanúsítvánnyal, vagy ACL-szabály hiányában próbál például az *iot/lab/sensor1/temperature* topicra publikálni, a broker naplója szerint elutasításra kerülne.

Fontos kiemelni, hogy a dolgozatban bemutatott mérések ténylegesen a Konfiguráció A és B környezetében valósultak meg; a Konfiguráció C jelenleg **koncepcionális architektúrát** képvisel. A fenti mTLS + ACL beállítások implementálása és tesztelése egyértelmű továbblépési irányt jelöl ki: a 7. fejezetben bemutatott jogosulatlan publish scénáriók újrafuttatása egy ilyen szigorúbb brokerkonfigurációval jól mérhetővé tenné, hogy milyen mértékben csökken a támadási felület, és milyen további többletterhelést jelent a rendszer számára az erős kliensazonosítás és az ACL-alapú hozzáférés-vezérlés.

7. Támadási scénáriók, tesztelés és mérési eredmények – II.

Ebben a fejezetben a 5.3.4 alfejezetben definiált támadási scénáriók gyakorlati megvalósítását és hatását vizsgálom a 6. fejezetben bemutatott szimulációs környezetben. A cél annak bemutatása, hogy a különböző biztonsági szintek – titkosítatlan MQTT, TLS 1.3, valamint a tervezett mTLS + ACL – milyen módon befolyásolják a rendszer viselkedését normál működés, illetve támadás alatt.

A 7. fejezet felépítése a következő logikát követi. Először (7.1) áttekintem, hogyan viselkedik a rendszer biztonságos konfigurációban, támadások nélkül, majd a 7.2–7.4 alfejezetekben konkrét jogosulatlan publish kísérleteken keresztül vizsgálom a védelmi mechanizmusok

hatását. A 7.5 alfejezetben a különböző konfigurációk eredményeit összefoglaló módon hasonlítom össze.

7.1 Normál működés biztonságos konfigurációban

Ebben az alfejezetben azt vizsgálom, hogyan működik a szimulált IoT-rendszer **támadások nélkül**, amikor a broker **TLS 1.3 feletti MQTT-t** használ (Konfiguráció B), és a szenzorok „egészséges”, normál üzemmódban küldik az adatokat. Ez a kiindulási referenciapont a később bemutatott támadási scénáriókhoz: csak akkor értelmezhetők jól a manipulált futások, ha ismert, hogyan néz ki egy stabil, biztonságos működés.

7.1.1 Szenárió és környezet rövid áttekintése

A normál, biztonságos működés vizsgálatához a következő beállításokat alkalmaztam:

- **Brokerkonfiguráció:**
 - Mosquitto broker TLS 1.3-mal, a 4.2.2 és 6.3 alfejezetben bemutatott beállítások szerint.
 - A broker a 8883-as porton fogadja az MQTT-over-TLS kapcsolatokat.
 - A szerver tanúsítványát egy saját CA (ca.crt) írta alá, a kliensek a CA-tanúsítvány segítségével ellenőrzik a broker identitását.
- **Szenzorok (Wokwi + ESP32):**
 - Két szimulált ESP32 node (sensor1 és sensor2), a 6.2 alfejezetben ismertetett kódbázissal.
 - Mindkét szenzor Wi-Fi-n keresztül csatlakozik, majd TLS-es MQTT-kapcsolatot hoz létre a brokerrel.
 - A szenzorok fix periódussal (a kódban beállított delay intervallummal) publikálnak hőmérsékleti értékeket a következő topicokra:
 - `iot/lab/sensor1/temperature`
 - `iot/lab/sensor2/temperature`
- **Mérési backend:**
 - A `collector_tls.py` program a TLS-es brokerhez csatlakozik, és feliratkozik a fenti topicokra.
 - Minden beérkező üzenetet időbélyeggel együtt rögzít a `measurements_tls.csv` állományba.
 - Az `analyze_measurements_tls.py` szkript feldolgozza a CSV-t, és szenzoronként kiszámítja az 5.3.2 alfejezetben definiált mérőszámokat (üzenetszám, időtartam, üzenetküldési ráta, átlaghőmérséklet).

A mérés során **nincs támadó kliens** a rendszerben: csak a két szenzor és a collector kommunikál a brokerrel. Így jól látható, hogyan viselkedik a rendszer, ha a forgalom „tiszta”, és a TLS egyetlen szerepe a csatorna titkosítása és a szerverhitelesítés.

7.1.2 Üzenetküldési ráta és időbeli viselkedés

A TLS-es normál futásokból származó measurements_tls.csv állomány feldolgozása után az analyzer szkript szenzoronként a következőket számolja ki:

- **Üzenetszám N_{msg} :** hány darab hőmérsékleti üzenet érkezett az adott szenzortól a mérési időablak alatt.
- **Időtartam T_{run} :** az első és utolsó üzenet időbélyege közötti különbség másodpercenként.
- **Becsült üzenetküldési ráta λ :** $N_{\text{msg}} / T_{\text{run}}$ hányad, azaz üzenetek száma másodpercenként.

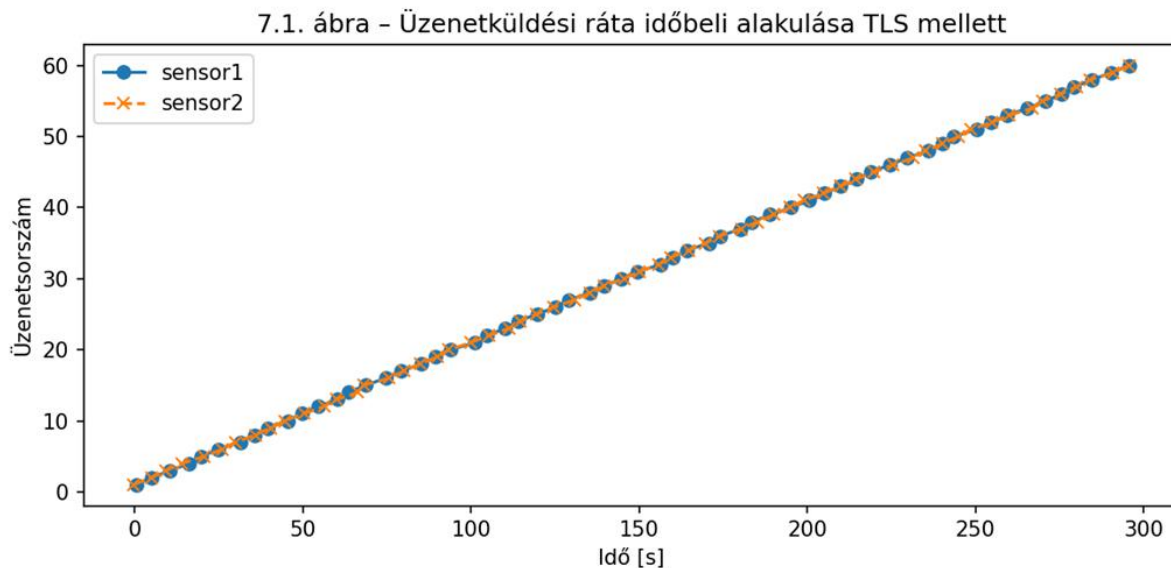
A 5.4.2. táblázatban bemutatott TLS-es mérési eredmények azt mutatják, hogy mindkét szenzor esetében:

- az üzenetszám **a várt nagyságrendben** alakul (a mérési időablakhoz és a kódban beállított periódushoz képest),
- az üzenetküldési ráta **stabil**, csak kisebb fluktuáció figyelhető meg,
- nem tapasztalható tömeges üzenetvesztés vagy összeomlás.

A 7.1. ábra – **Üzenetküldési ráta időbeli alakulása TLS mellett** (a measurements_tls.csv adatai alapján készített diagram) szemléletesen mutatja, hogy a szenzorok által küldött üzenetek időben egyenletesen érkeznek. Az esetleges néhány másodperces ingadozások elsősorban:

- az ESP32-es kód időzítésének pontatlanságából,
- a Wokwi szimuláció futtatási ütemezéséből,
- a hálózati stack belső puffereeléséből

adódnak, és nem jelentenek lényegi problémát a rendszer megbízhatósága szempontjából.



Forrás: saját szerkesztés

A normál, biztonságos működés szempontjából az a legfontosabb, hogy:

- a szenzorok hosszabb időn keresztül is **folyamatosan és kiszámítható ütemben** tudnak adatot küldeni,
- a TLS használata nem okoz olyan mértékű többlet késleltetést vagy instabilitást, ami a funkcionális működést veszélyeztetné.

7.1.3 Hőmérsékleti értékek eloszlása és konzisztenciája

A hőmérsékleti adatok a szimulációban valójában **véletlenszámok**, azonban a rendszer szempontjából úgy kezelhetők, mintha valós szenzormérések lennének. Az `analyze_measurements_tls.py` az alábbi statisztikákat számítja ki szenzoronként:

- **Átlagos hőmérséklet $\bar{T}$ [°C]:** a mért értékek számtani közepe,
- (opcionálisan) **szórás, minimum, maximum**, amelyek a hőmérsékleti tartományt írják le.

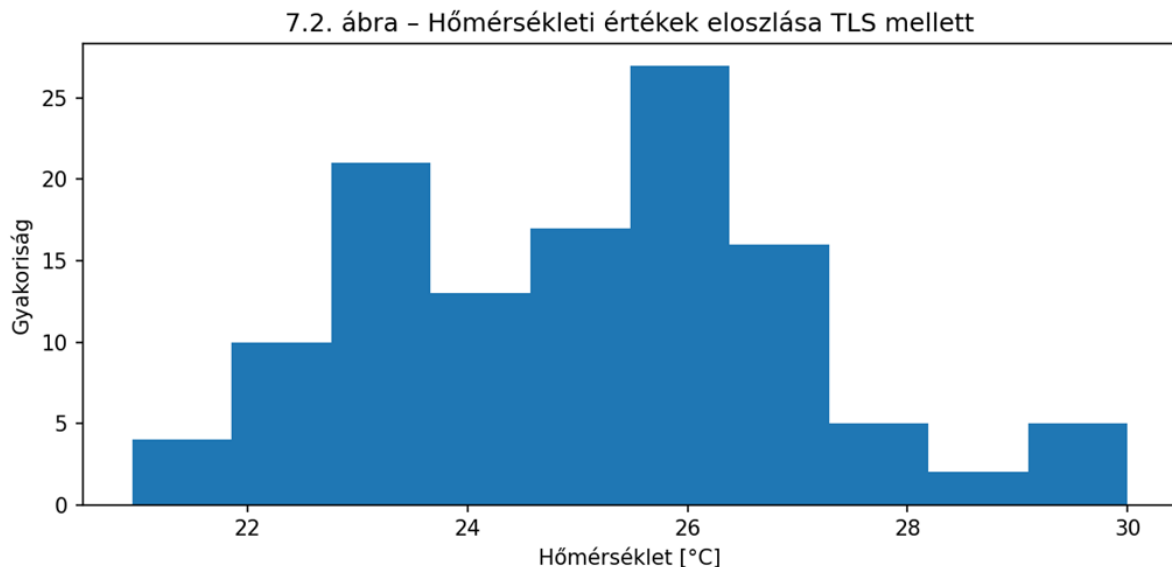
A TLS-es normál futás során kapott átlaghőmérsékleti értékek:

- **a 20–30 °C közötti intervallumba esnek,**
- a két szenzor között csak kisebb eltérések figyelhetők meg, ami megfelel a véletlenszerű generálásból adódó különbségeknek.

A VII.II. ábra – Hőmérsékleti értékek eloszlása TLS mellett (hisztogram vagy boxplot a `measurements_tls.csv` alapján) azt mutatja, hogy a mért adatok:

- nem tartalmaznak extrém kiugró (például nagyon magas vagy nagyon alacsony, irreális) értékeket,

- nagyjából egyenletesen oszlanak el a beállított intervallumon belül,
- formailag épek (érvényes JSON payload, helyes típusok).



Forrás: saját szerkesztés

Ez arra utal, hogy:

- a TLS bevezetése **nem okozott adatvesztést vagy payload-torzulást**,
- a collector_tls.py a titkosított csatorna ellenére ugyanúgy képes az üzenetek megbízható naplózására, mint a titkosítatlan collector.py,
- a feldolgozott statisztikák alkalmasak arra, hogy később összehasonlítsuk őket a támadások alatt rögzített adatokkal.

7.1.4 Biztonsági megfigyelések és következtetések

A normál, biztonságos TLS-es futások alapján az alábbi következtetések vonhatók le:

- **Csatornaszintű** **biztonság:**
A TLS 1.3 gondoskodik arról, hogy a szenzorok és a broker közötti forgalom titkosított legyen. A lehallgatás és a man-in-the-middle típusú támadások jelentősen nehezebbé válnak, mivel a támadó nem fér hozzá a tiszta szövegű hőmérsékleti adatokhoz.
- **Szerverhitelesítés:**
A collector_tls.py a ca.crt segítségével ellenőrzi, hogy valóban a megbízható brokerhez csatlakozik. Ezzel elkerülhető, hogy a szenzorok és a backend egy hamis, támadó által üzemeltetett brokerrel kommunikáljanak.

- **Funkcionális**

megfelelőség:

A mért üzenetküldési ráták és az átlaghőmérsékletek azt mutatják, hogy a TLS bevezetése **nem rontja érdemben a rendszer funkcionalitását:**

- a szenzorok stabilan tudnak adatot küldeni,
- az analyzer által számolt mutatók konzisztens képet adnak a rendszer állapotáról.

Ugyanakkor fontos hangsúlyozni, hogy:

- a TLS **elsősorban a csatorna biztonságát** garantálja,
- **nem dönt arról**, hogy ki *milyen topicokra* publikálhat vagy iratkozhat fel.

Ez azt jelenti, hogy ha egy támadó kliens ismeri a broker címét és rendelkezik a CA-tanúsítvánnyal, technikailag ő is létre tud hozni egy titkosított kapcsolatot, és – megfelelő ACL-ek hiányában – hamis adatokkal terhelheti a szenzor-topicokat. A 7.2 és 7.3 alfejezetek éppen ezt a problémát illusztrálják: megmutatják, hogy titkosítás nélkül és pusztán TLS mellett milyen könnyen megvalósítható a **jogosulatlan publish**, és hogyan torzítja ez a collector által mért statisztikákat.

7.2 Jogosulatlan publish titkosítatlan MQTT-n (Konfiguráció A)

Ebben az alfejezetben azt vizsgálom, hogy a baseline, titkosítatlan MQTT-konfigurációban (Konfiguráció A) milyen egyszerűen valósítható meg egy **jogosulatlan publish** támadás, és ez hogyan jelenik meg a collector és az analyzer nézőpontjából. A cél nem kifinomult támadások modellezése, hanem annak demonstrálása, hogy egy anonim MQTT-broker – megfelelő hitelesítés és ACL-ek hiányában – a legprimitívebb eszközökkel is manipulálható.

7.2.1 Támadási modell és cél

A támadó modellje a következő:

- technikailag felkészült, de nem feltétlenül „állami szintű” támadó,
- rendelkezik internetkapcsolattal, és ismeri a publikus MQTT-broker (test.mosquitto.org) címét,
- szabadon használhat saját MQTT-klienst (például egy egyszerű Python-szkriptet a paho-mqtt könyvtárral).

A baseline Konfiguráció A esetében a broker:

- titkosítatlan MQTT-t használ a 1883-as porton,
- nem követel meg felhasználónév/jelszó párost,
- nem alkalmaz topic-szintű hozzáférés-szabályozást (ACL-t).

Ennek következtében a támadó **ugyanarra a brokerre** képes csatlakozni, mint a legitim szenzorok, és – ha ismeri a topic nevét – **ugyanabba a topicba** tud publish-elni, mint például a sensor1:

- legitim topic: iot/lab/sensor1/temperature.

A támadás célja ebben a szcenárióban az, hogy:

- a sensor1 által küldött hőmérsékleti adatokat „megmérgezze” (data poisoning),
- a collector által mért statisztikákat (N_msg, T_run, átlaghőmérséklet) eltolja egy hamis irányba,
- mindezt úgy, hogy a collector és az analyzer **önmagukban** ne tudják megkülönböztetni a legitim és a támadó által küldött üzeneteket.

7.2.2 A támadó kliens implementációja (pub_test.py)

A támadó szerepét a gyakorlatban egy egyszerű Python-szkript tölti be, amely a 6. fejezetben bemutatott tesztkliens (pub_test.py) módosított változatára épül. A szkript a következő lépéseket hajtja végre:

1. Kapcsolódás a publikus brokerhez:

- broker cím: test.mosquitto.org,
- port: 1883,
- titkosítás nélkül, anonim módon (nincs felhasználónév/jelszó).

2. Topic kiválasztása:

- a támadó direkt a legitim sensor1 topicjára publikál:
 - iot/lab/sensor1/temperature.

3. Payload generálása:

- a JSON payload mezői formailag azonosak a legitim szenzor üzeneteivel, például:

```
{  
  "sensor_id": "sensor1",  
  "ts_device": 123456,  
  "temperature": 80  
}
```

Forrás: saját szerkesztés

- a különbség az, hogy a temperature mező értéke tudatosan **irreális** (pl. 70–80 °C), vagy a legitim szenzorhoz képest lényegesen eltérő tartományban mozog.

4. Üzenetküldési stratégia:

- o a támadó a legitim szenzoroknál **gyakrabban** küld üzenetet (például 1–2 másodperces periódussal),
- o ezzel az a célja, hogy a collector által mért üzenetszámot (N_{msg}) és átlaghőmérsékletet ($\bar{T}$) dominálja, eltolva a statisztikákat a hamis értékek irányába.

A támadó kliens szemszögéből a csatlakozás menete gyakorlatilag egy normál MQTT-kliens működésével egyezik meg: a broker **nem különbözteti meg** a legitim és a támadó klienst, és nem ismer olyan policy-t, amely tiltja, hogy bárki a sensor1 topicjára írjon.

7.2.3 Hatás a collector és analyzer nézőpontjából

A jogosulatlan publish támadás hatásának megértéséhez kulcsfontosságú, hogy a collector és az analyzer **nem lát bele** a hálózati rétegbe, és nincs közvetlen információjuk arról, hogy egy üzenet melyik konkrét eszköztől érkezett. Számukra:

- minden üzenet, amely az iot/lab/sensor1/temperature topicon érkezik,
- és formailag érvényes JSON payloadot tartalmaz,

„**sensor1 adata**” – függetlenül attól, hogy azt valójában a Wokwi-ban futó ESP32 vagy a támadó Python-szkript küldte.

Egy tipikus támadott baseline futás measurements.csv fájlja ezért:

- a sensor1 esetében **jóval magasabb N_{msg} értéket** tartalmaz, mint normál, támadás nélküli futáskor,
- az átlaghőmérséklet $\bar{T}$ értéke **eltolódik** a támadó által küldött irreális értékek irányába (például 24–25 °C helyett 35–40 °C körüli tartományba),
- az időbeli eloszlásban (időbélyegek vizsgálatakor) sűrűbb „burst”-ök, rövidebb üzenetközi idők figyelhetők meg.

A collector és analyzer szemszögéből tehát a támadás eredménye:

- **nem technikai hiba** (nem omlik össze a rendszer),
- hanem **adatminőségi probléma**: a mért statisztikák torzulnak, a hőmérsékleti profil teljesen más képet mutat a valósághoz képest.

Ez különösen IoT-alapú döntéstámogató rendszerekben veszélyes, ahol az automatizált szabályok (például riasztások, vezérlési parancsok) közvetlenül a szenzoradatokra támaszkodnak.

7.2.4 Biztonsági következtetések titkosítatlan MQTT mellett

A titkosítatlan, Konfiguráció A szerinti MQTT-környezetben a fenti támadási szcenárió könnyen kivitelezhető, és több fontos tanulságra világít rá:

- **Nincs** **kliensidentitás:**
A broker nem különbözteti meg, hogy ki küldte az üzenetet. Ha egy üzenet az `iot/lab/sensor1/temperature` topicba érkezik, a rendszer nem tudja megmondani, hogy az a „valódi” `sensor1`-től vagy egy külső támadótól származik-e.
- **Nincs** **hozzáférés-szabályozás:**
ACL-ek hiányában bárki írhat bármely topicra. A támadó ugyanazt a `publish`-jogosultságot élvezi, mint a legitimen konfigurált szenzor.
- **Nincs** **csatornaszintű** **védelem:**
A forgalom titkosítatlanul halad át, így a jogosulatlan `publish` mellett a lehallgatás és a `man-in-the-middle` támadások is triviálisan kivitelezhetők.
- **Detektálás** **nehézsége:**
Ha a támadó `payloadja` formailag érvényes (JSON, megfelelő mezőnevek, ésszerű típusok), akkor a `collector/analyzer` réteg kizárólag az alkalmazáslogika szintjén – például anomáliadetektálással – tudná észrevenni a problémát. A jelen dolgozat mérési keretrendszere ilyen automatizált detekciót még nem valósít meg, csak a torzult statisztikák alapján következtet a támadás sikerességére.

Összességében a Konfiguráció A vizsgálata rávilágít arra, hogy egy egyszerű, anonim MQTT-broker használata **komoly kockázatot jelent**, még akkor is, ha a szenzorok kódja hibátlanul működik. A rendszer integritása nemcsak attól függ, hogy „mit” küld a szenzor, hanem attól is, hogy **ki mindenki képes ugyanabba a csatornába hamis információt injektálni**. A következő alfejezetek éppen ezért azt vizsgálják, hogy ugyanez a támadás TLS-sel (Konfiguráció B), illetve a tervezett mTLS + ACL bővítés mellett (Konfiguráció C) milyen mértékben korlátozható vagy teljesen blokkolható.

7.3 Jogosulatlan `publish` TLS felett (Konfiguráció B)

A 7.2 alfejezetben bemutattam, hogy a titkosítatlan, anonim MQTT-környezetben (Konfiguráció A) egy támadó kliens gyakorlatilag akadálytalanul képes jogosulatlan `publish` műveleteket végrehajtani a szenzorok topicjaira. Ebben az alfejezetben azt vizsgálom meg,

hogyan **ugyanaz a támadás TLS-es környezetben (Konfiguráció B)** milyen formában valósítható meg, és miben tér el a hatása az előző esethez képest.

7.3.1 Támadási modell TLS-es környezetben

Konfiguráció B-ben a Mosquitto broker TLS 1.3 feletti MQTT-t használ a 8883-as porton. A broker:

- szervertanúsítványt mutat be, amelyet egy saját CA (ca.crt) írt alá,
- a kliensek a ca.crt alapján ellenőrzik, hogy valóban a megbízható brokerhez kapcsolódnak,
- továbbra is engedélyezi az anonim hozzáférést (nincs kötelező felhasználónév/jelszó),
- topic-szintű ACL-ek nincsenek beállítva.

A **támadó modellje** ebből a szempontból hasonló a 7.2-ben vizsgált esethez:

- ismeri a broker címét (jelen esetben tipikusan localhost, lokális futtatás mellett),
- birtokában van a CA-tanúsítványnak (ca.crt),
- képes TLS-es MQTT-kapcsolatot létrehozni a brokerrel (pl. a paho-mqtt TLS funkcióival).

A támadó kliens tehát TLS-en keresztül csatlakozik a brokerhez, és ugyanazt a topicot célozza meg, mint korábban:

- `iot/lab/sensor1/temperature`.

A fő különbség a Konfiguráció A-hoz képest az, hogy itt a kommunikációs csatorna **titkosított**, és a támadó csak akkor tud csatlakozni, ha rendelkezik a megbízható CA tanúsítványával. Amennyiben azonban ezt megszerzi (például azért, mert a CA-tanúsítványt a fejlesztői dokumentáció részeként megosztják), a broker továbbra sem tud különbséget tenni a legitim szenzor és a támadó kliens között.

7.3.2 A támadó kliens TLS-es változata

A gyakorlatban a támadó kliens a 6.3 alfejezetben bemutatott `pub_test.py` szkript TLS-esített változataként képzelhető el (pl. `pub_test_tls.py`). A műveletek lényegében az alábbiak:

1. TLS-es kapcsolat felépítése a brokerrel:

- broker: localhost,
- port: 8883,
- TLS konfiguráció a paho-mqtt oldalon:
 - `client.tls_set(ca_certs="ca.crt", ...)`,
 - opcionálisan szigorú tanúsítványellenőrzés beállítása.

2. Azonos topic használata, mint a legitim szenzor:

- `iot/lab/sensor1/temperature`.

3. Payload generálása:

- formailag érvényes JSON, például:

```
{
  "sensor_id": "sensor1",
  "ts_device": 987654,
  "temperature": 75
}
```

Forrás: saját szerkesztés

- a `sensor_id` mező értékét a támadó szándékosan "sensor1"-re állítja, hogy a collector és analyzer szemében a támadó üzenetei is „sensor1” adataiként jelenjenek meg.

1. Támadási stratégia:

- a támadó a legitim szenzoroknál **gyakrabban** küld üzenetet (pl. 1 s periódussal),
- a hőmérséklet mezőben extrém értékeket (70–80 °C) vagy szokatlan mintázatot alkalmaz,
- célja, hogy a TLS-es futás `measurements_tls.csv` fájljában a `sensor1`-hez tartozó sorok jelentős része az ő hamis adatait tartalmazza.

A broker szempontjából ez a kliens ugyanolyan, mint bármely más TLS-es kliens: sikeres TLS handshake után jogosult MQTT-kommunikációt folytatni, és mivel ACL-ek nincsenek, a `sensor1` topicjára történő publish sem ütközik korlátozásba.

7.3.3 A támadás hatása a TLS-es mérési adatokra

A `collector_tls.py` és az `analyze_measurements_tls.py` szemszögéből a támadás hatása meglepően hasonló a titkosítatlan Konfiguráció A-hoz:

- a `measurements_tls.csv` állományban a **sensor1** sorai között megjelennek a támadó által küldött üzenetek is,
- a `sensor_id` mező mindkét esetben "sensor1", így a backend nem tudja megkülönböztetni a forrásokat,
- az időbélyegek alapján a `sensor1`-hez tartozó üzenetek **sűrűbb ütemben** érkeznek, mint egy normál, támadás nélküli futásban,
- az átlaghőmérséklet $\bar{T}$ a `sensor1` esetében eltolódik a támadó által megadott extrém értékek felé.

Ha egy normál, biztonságos TLS-es futásban a sensor1 átlaghőmérséklete például ~25 °C körül alakul, egy támadott futásban könnyen elképzelhető, hogy:

- az átlag 30–35 °C fölé tolódik,
- az üzenetküldési ráta $\lambda_{sensor1}$ megnő, mivel a támadó rövidebb periódussal küld adatot, mint a legitim szenzor.

A rendszer szempontjából a lényeg ugyanaz, mint a 7.2-es esetben:

- **nem jelenik meg hiba**, a collector és analyzer továbbra is működik,
- a keletkező statisztikák azonban **nem a valós fizikai állapotot tükrözik**, hanem egy támadó által manipulált mintázatot.

A különbség „csak” annyi, hogy itt mindez egy **titkosított csatornán** történik, tehát a külső lehallgatók számára a forgalom tartalma rejtve marad. Ez jól mutatja, hogy a TLS önmagában a **bizalmasságot**, nem pedig a **forrás megbízhatóságát** garantálja.

7.3.4 Biztonsági tanulságok TLS mellett

A jogosulatlan publish TLS felett történő vizsgálata több fontos tanulságot hoz:

- **TLS** $\neq$ **teljes biztonság:**
A TLS elsődleges célja a csatornaszintű titkosítás és a szerver hitelesítése. Ha a kliensek számára nincs erős identitáskezelés (mTLS), és nincsenek topic-szintű ACL-ek, a támadó kiválóan tud „bújni” egy legitim kliens álarca mögé, csak éppen titkosított csatornán.
- **Azonosítás** **hiánya:**
A TLS-es kapcsolatfelépítés során – mTLS hiányában – a broker nem kap megbízható információt arról, hogy „ki” a kliens, csak arról, hogy a kommunikáció egy adott CA által aláírt tanúsítvánnyal rendelkező szerverhez zajlik. Ez a szerverhitelesítés fontos, de nem elég a jogosulatlan kliensek kiszűréséhez.
- **ACL-ek** **hiánya:**
Ha nincsenek beállítva topic-szintű ACL-ek, a broker nem tudja megakadályozni, hogy egy tetszőleges kliens az `iot/lab/sensor1/temperature` topicra írjon. A támadás így ugyanúgy sikeres, mint a titkosítatlan esetben, csak a lehallgatás nehezebb.
- **Detektálás** **továbbra is nehéz:**
A TLS nem teszi könnyebbé a támadás detektálását a backend számára. A collector és analyzer továbbra is csak az üzenetek tartalmát látja (amik formailag érvényesek), a transport-szintű metaadatokat nem. A támadás észlelése továbbra is valamilyen

anomáliadetektáló logikát igényelne (például hirtelen megugró üzenetküldési ráta vagy irreális hőmérsékleti értékek alapján).

A Konfiguráció B vizsgálata tehát egyértelműen jelzi, hogy a TLS bevezetése **szükséges, de nem elégséges** lépés az IoT-rendszerek védelmében. A jogosulatlan publish elleni védelemhez:

- **kliensoldali hitelesítésre (mTLS)** van szükség, hogy a broker egyértelműen azonosítani tudja a szenzorokat és a többi klienst,
- **ACL-ekre** van szükség, hogy a kliensidentitás alapján szigorúan szabályozható legyen, ki mely topicokra publikálhat és melyekre iratkozhat fel.

A 6.3 alfejezetben vázolt Konfiguráció C (mTLS + ACL) éppen ennek a továbbfejlesztett biztonsági modellnek a koncepcionális alapját adja. A 7.4 alfejezetben ennek a megoldásnak a várható hatását, illetve lehetséges implementációs stratégiáit tekintem át.

7.4 mTLS + ACL alapú védelem várható hatása (Konfiguráció C – koncepcionális elemzés)

A 7.2 és 7.3 alfejezetekben bemutatott támadási scenáriók alapján látható, hogy sem a titkosítatlan MQTT (Konfiguráció A), sem a pusztán TLS-sel védett MQTT (Konfiguráció B) nem ad teljes körű védelmet a jogosulatlan publish műveletekkel szemben. A támadó mindkét esetben képes volt a legitim szenzorok topicjaira hamis adatokat küldeni, és ezzel a collector által mért statisztikákat torzítani.

A Konfiguráció C célja ennek a hiányosságnak a kezelése: a csatornaszintű titkosításra (TLS 1.3) építve **kölcsönös hitelesítéssel (mTLS)** és **topic-szintű hozzáférés-vezérléssel (ACL-ekkel)** védi a brokert. Mivel a dolgozat gyakorlati része a teljes mTLS + ACL megoldást már nem implementálja végig, ebben az alfejezetben a védelem **koncepcionális hatását** elemzem, a 6.3 alfejezetben vázolt Mosquitto-konfigurációra támaszkodva.

7.4.1 Védelmi modell: kliensidentitás és minimális jogosultság

A Konfiguráció C alapgondolata, hogy a broker csak olyan kliensekkel hajlandó kommunikálni, amelyek:

1. **érvényes, megbízható CA által aláírt kliens-tanúsítvánnyal** rendelkeznek (mTLS), és
2. **kifejezetten engedélyezett topicokra** próbálnak publish/subscribe műveletet végrehajtani (ACL).

A 6.3 alfejezetben bemutatott Mosquitto-konfiguráció ennek megfelelően:

- `require_certificate true` beállítással megköveteli a kliens-tanúsítvány bemutatását,

- a `use_identity_as_username true` opció segítségével a tanúsítványban szereplő identitást (például CN mező) használja „felhasználónévként”,
- az `allow_anonymous false` opcióval letiltja az anonim kapcsolódást,
- az `acl_file` állományban pedig topic-szintű szabályokat rögzít, például:

```
user sensor1
topic write iot/lab/sensor1/temperature

user sensor2
topic write iot/lab/sensor2/temperature

user collector
topic read iot/lab/#
```

Forrás: saját szerkesztés

Ebben a modellben a szenzorok, a collector és az esetleges adminisztrátori kliensek **mind saját tanúsítvánnyal rendelkeznek**, amelyet a központi CA állít ki. A broker az mTLS handshake során ebből a tanúsítványból azonosítja a klienst, és az ACL-ek alapján dönt arról, hogy milyen műveletek engedélyezettek számára.

A **minimális jogosultság elve (principle of least privilege)** itt konkrét formában jelenik meg:

- a `sensor1` csak az `iot/lab/sensor1/temperature` topicra írhat,
- a `sensor2` csak az `iot/lab/sensor2/temperature` topicra írhat,
- a `collector` csak olvas (subscribe) jogosultságot kap az összes lab topicra,
- más topicra (pl. `iot/lab/sensor1/temperature` helyett `iot/lab/admin/cmd`) egyik szenzor sem írhat, amennyiben az ACL ezt nem engedi.

7.4.2 A jogosulatlan publish várható kimenetele mTLS + ACL mellett

Ha a 7.2 és 7.3 alfejezetben bemutatott támadó klienst változtatás nélkül próbálnánk használni egy ilyen mTLS + ACL-es broker ellen, több védelmi mechanizmus is életbe lépne:

1. Kliens-tanúsítvány hiánya (mTLS):

- Amennyiben a támadó nem rendelkezik a CA által aláírt, érvényes kliens-tanúsítvánnyal, a TLS handshake már a kapcsolatfelépítés során meghiúsul.
- A Mosquitto naplójában ez jellemzően sikertelen handshake kísérletként jelenik meg, a broker nem engedélyez MQTT-szintű kommunikációt.

2. Hibás vagy ismeretlen identitás:

- Ha a támadó valamilyen módon mégis kliens-tanúsítványt szerez (például egy másik eszköz kompromittálásával), de a tanúsítványban szereplő CN / identity

nem szerepel az ACL-ben, a broker ugyan felépítheti a TLS kapcsolatot, de az MQTT műveletek (publish/subscribe) **tiltásra kerülnek**.

- Az ACL-szabályzatból hiányzó felhasználó esetén a Mosquitto a vonatkozó topicokra vonatkozóan not authorized hibát ad.

3. Topic-szintű tiltás:

- Ha a támadó valahogy megszerezne egy létező felhasználónak megfelelő kliens-tanúsítványt (például egy kompromittált sensor2-t), akkor is csak az ACL-ben engedélyezett topicokra írhatna.
- Egy sensor2 identitással rendelkező kliens például **nem** publikálhatna az `iot/lab/sensor1/temperature` topicra, mivel az ACL ezt nem engedi.

A Konfiguráció C-ben tehát a 7.2–7.3-ban bemutatott jogosulatlan publish kísérletek **három, egymásra épülő szűrőn is fennakadnának**:

- kliens-tanúsítvány ellenőrzése,
- identity → felhasználónév leképezés,
- ACL alapú topic-hozzáférés ellenőrzése.

A rendszer viselkedése támadás alatt a gyakorlatban a következőképpen nézne ki:

- a legitim szenzorok továbbra is zavartalanul tudnának adatot küldeni a saját topicjukra,
- a támadó klientsztől érkező kapcsolódási / publish kísérletek vagy már a TLS handshake során megghiúsulnának,
- vagy MQTT-szinten azonnal elutasításra kerülnének (not authorized),
- a `measurements.csv` / `measurements_tls.csv` állományokban **nem jelenne meg** egyetlen sikeres, támadó által küldött hőmérsékleti adat sem.

Ha bevezetnénk egy kiegészítő metrikát (például N_{denied} : elutasított publish/subscribe kísérletek száma), akkor Konfiguráció C alatt támadás során:

- a legitim szenzorokra vonatkozó N_{msg} változatlan maradna,
- a támadóhoz köthető N_{denied} érték viszont megnőne, amit a broker naplói is alátámasztanának.

7.4.3 Implementációs szempontok és korlátok

Bár a Konfiguráció C erős védelmi modellt képvisel, gyakorlati bevezetése több kihívással és trade-offal jár:

- **Tanúsítványkezelés** **(PKI):**
A mTLS használata szükségessé teszi egy belső PKI (tanúsítványkiadó és -kezelő infrastruktúra) kialakítását. Nagyszámú IoT-eszköz esetén ez jelentős adminisztratív és

üzemeltetési feladatot jelent (tanúsítványok kiadása, megújítása, visszavonása, biztonságos tárolása az eszközökön).

• **ACL-ek** **karbantartása:**

Az ACL-állomány(ok) naprakészen tartása – különösen dinamikusan változó, nagy szenzorpark esetén – szintén nem triviális feladat. Hibás ACL-konfiguráció esetén könnyen előfordulhat, hogy:

- legitim szenzorokat zárunk ki (téves tiltás), vagy
- támadók számára véletlenül nyitva hagyunk bizonyos topicokat.

• **Erőforráskorlátok** **az** **eszközökön:**

Az mTLS kliensoldali implementációja számításigényesebb, mint egy egyszerű, titkosítatlan MQTT-kliens. Bár az ESP32-szintű hardverek általában képesek TLS-t kezelni, nagyon erősen erőforrás-korlátozott IoT-eszközök esetén ez problémát jelenthet.

• **Komplexebb** **hibakeresés** **és** **üzemeltetés:**

A tanúsítványok érvényességi ideje, a lejáratok, a CA-k cseréje és az ACL-ben lévő finom szabályok a rendszer üzemeltetését komplexebbé teszik. Ha valami nem működik (például egy szenzor „váratlanul elnémul”), a hiba oka lehet egy lejárt tanúsítvány, egy hibás ACL-sor, vagy egy rosszul konfigurált kliens is.

A dolgozat gyakorlati keretei között a teljes mTLS + ACL megoldás végigvezetése és mérések validálása már nem fért bele, ugyanakkor a 6.3 és 7.4 alfejezetekben bemutatott architektúra és koncepcionális elemzés világosan jelzi:

- **milyen irányba érdemes továbbfejleszteni** egy jelenlegi, TLS-t már használó, de kliensidentitást nem kezelő IoT-rendszert,
- **milyen típusú támadások ellen** nyújt védelmet az mTLS + ACL kombináció (jogosulatlan publish, topic-szintű visszaélés),
- **és milyen új problémákat** vet fel (tanúsítványkezelés, ACL-adminisztráció, erőforrásigény).

Összességében elmondható, hogy a Konfiguráció C a 2. fejezetben tárgyalt Zero Trust elvek gyakorlati, MQTT-alapú megvalósításának irányába mutat: a broker már nem feltételez implicit bizalmat egyetlen klienssel szemben sem, a hozzáférés minden esetben explicit, tanúsítvány- és ACL-alapú döntés eredménye. Ez a modell – a dolgozat hipotézise szerint – jelentősen csökkentené a 7.2–7.3-ban bemutatott támadási scénáriók sikerességi esélyét, ugyanakkor a további kutatás feladata marad annak részletes, mérés-alapú vizsgálata, hogy mindez milyen többletterhelést ró a rendszerre egy nagyléptékű IoT-deploy esetén.

7.5 MI-alapú anomáliadetektálási kísérlet

A 7.2–7.4 alfejezetekben bemutatott támadási scenáriók azt igazolták, hogy a csatornaszintű védelem és a hozzáférés-vezérlés mellett a forgalom viselkedésének folyamatos monitorozása is fontos szerepet játszhat az IoT-rendszerek biztonságában. Ennek szemléltetésére a dolgozat gyakorlati részében egy egyszerű, gépi tanulás alapú anomáliadetektálási kísérlet is készült, amelynek célja annak vizsgálata volt, hogy a normál és támadó MQTT-forgalom mennyire különíthető el automatikus módszerekkel.

A detektáló modul Python környezetben, az Isolation Forest algoritmus felhasználásával került kialakításra. A modell tanítóhalmaza a legitim szenzorok normál forgalmát tartalmazó `measurements_normal.csv` állomány volt, míg a teszteléshez egy olyan `measurements_attack.csv` állomány készült, amely a normál szenzoradatok mellett külön `attack` azonosítóval ellátott, ugyanarra a topicstruktúrára publikált támadó üzeneteket is tartalmazott. A támadó kliens a kísérlet során a legitim szenzorok által használt MQTT-topicra küldött hamis értékeket, azonban a payloadban elkülöníthető `attack sensor_id`-vel szerepelt.

A feature-képzés során három jellemző került felhasználásra: a szenzorazonosító numerikus reprezentációja, a hőmérsékleti érték, valamint az ugyanattól a szenzortól érkező egymást követő üzenetek közötti időeltérés. A normál és támadó minták elkülönítése ezek alapján történt. A tesztállomány összesen 232 mintát tartalmazott, ebből 120 normál és 112 támadó mintát.

A modell teljesítményének kiértékelése során a confusion matrix a következő eredményt adta:

$$\begin{bmatrix} 116 & 4 \\ 0 & 112 \end{bmatrix}$$

Az eredmény azt mutatja, hogy a modell 116 normál mintát helyesen sorolt a normál osztályba, 4 normál mintát tévesen anomáliának jelölt, ugyanakkor egyetlen támadó mintát sem hagyott észrevétlenül, és mind a 112 támadó mintát sikeresen detektálta.

Az osztályozási mutatók alapján a támadó osztályra vonatkozóan a precision értéke 0,97, a recall értéke 1,00, az F1-score pedig 0,98 volt. A teljes teszhalmazra számított pontosság 0,98 lett. Különösen lényeges eredmény, hogy a recall értéke 1,00, vagyis a vizsgált adathalmazon a modell minden támadó mintát felismert. A téves riasztások száma alacsony maradt, ami arra utal, hogy a választott feature-készlet és az alkalmazott modell laboratóriumi környezetben jól elkülönítette a normál és a manipulált MQTT-forgalmat.

A kísérlet eredményei alapján megállapítható, hogy az MI-alapú anomáliadetektálás nem helyettesíti a hálózati és hozzáférés-vezérlési védelmet, ugyanakkor hatékony kiegészítő réteget jelenthet. Míg a TLS, az mTLS és az ACL-ek elsősorban a hozzáférés és a kommunikáció védelmét szolgálják, addig az anomáliadetektálás a már létrejött forgalom viselkedését elemzi, és segíthet olyan támadási mintázatok felismerésében is, amelyek pusztán protokollsinten nem mindig szűrhetők ki. A dolgozatban bemutatott egyszerű modell ezért jól illusztrálja, hogy az IoT-biztonságban a klasszikus és intelligens védelmi megoldások kombinációja eredményes megközelítést jelenthet.

7.6 Összefoglaló értékelés és konfigurációk összehasonlítása

A 7. fejezetben bemutatott vizsgálatok célja az volt, hogy a 2–4. fejezetben megfogalmazott elméleti megállapításokat konkrét, szimulált támadási scénáriókon keresztül is ellenőrizsem. A fókuszban az MQTT-alapú IoT-rendszerek egyik tipikus sebezhetősége, a **jogosulatlan publish** állt, valamint az, hogy a különböző biztonsági szintek – titkosítatlan MQTT, TLS 1.3, mTLS + ACL – hogyan befolyásolják a támadási felület nagyságát és a rendszer mérési eredményeit.

Az eredmények három fő konfiguráció mentén foglalhatók össze:

- **Konfiguráció A – titkosítatlan MQTT (baseline)**
- **Konfiguráció B – TLS 1.3 feletti MQTT (szerverhitelesítés)**
- **Konfiguráció C – mTLS + ACL (koncepcionális megoldás)**

7.6.1 Konfiguráció A: „nyitott” MQTT-busz

A titkosítatlan, anonim MQTT-környezetben (Konfiguráció A) a broker a 1883-as porton fogadta a kapcsolatokat, TLS nélkül, `allow_anonymous true` beállítással és ACL-ek nélkül. A 7.2 alfejezetben bemutatott jogosulatlan publish scenárió alapján az alábbi következtetések vonhatók le:

- A támadó kliens **rendkívül egyszerűen** tudott csatlakozni ugyanarra a brokerre, mint a legitim szenzorok. Ehhez elég volt ismernie a publikus broker címét és a topic nevét.
- A támadó formailag érvényes JSON-üzeneteket küldött az `iot/lab/sensor1/temperature` topicra, ugyanolyan `sensor_id` mezővel, mint a legitim `sensor1`.
- A collector és az analyzer **nem tudta megkülönböztetni** a legitim és a hamis üzeneteket; a `measurements.csv` állomány alapján a `sensor1`-re számított statisztikák (`N_msg`, átlaghőmérséklet) jelentősen eltorzultak.
- A rendszer „kívülről” továbbra is működőképesnek látszott, ugyanakkor az IoT-alapú döntéstámogatás szempontjából a mérési eredmények **megbízhatatlanná váltak**.

Ez a konfiguráció tehát egyfajta „**nyitott MQTT busz**” modelljét valósítja meg: funkcionálisan egyszerű és jól használható, de a biztonsági kockázatok miatt valós rendszerekben legfeljebb izolált, kísérleti környezetben tekinthető elfogadhatónak.

7.6.2 Konfiguráció B: TLS 1.3 – csatornavédelem, de korlátozott védelem a támadók ellen

A TLS 1.3-ra épülő Konfiguráció B bevezetése több szempontból jelentett előrelépést a baseline-hoz képest:

- A szenzorok és a broker közötti kommunikáció **titkosított csatornán** zajlott (8883-as port, saját CA által aláírt szervertanúsítvány).
- A `collector_tls.py` és a TLS-es analyzer bizonyította, hogy ilyen környezetben is stabilan rögzíthetők és feldolgozhatók az adatok (5.4.2 és 7.1 alfejezet).
- A csatornaszintű biztonság javult: a lehallgatási és man-in-the-middle típusú támadások lényegesen nehezebbé váltak.

Ugyanakkor a 7.3 alfejezetben vizsgált jogosulatlan publish scenárió rámutatott arra, hogy:

- a TLS **elsősorban a bizalmasságot és a szerverhitelesítést** biztosítja,
- önmagában **nem akadályozza meg**, hogy egy támadó kliens – ha ismeri a broker címét és rendelkezik a CA-tanúsítvánnyal – titkosított csatornán csatlakozzon,
- ACL-ek és mTLS hiányában a támadó továbbra is képes volt az `iot/lab/sensor1/temperature` topicra hamis adatokat publikálni, csak éppen titkosított formában.

A collector_tls és az analyzer szemszögéből a támadás hatása nagyon hasonló volt a Konfiguráció A-hoz: a sensor1 statisztikái eltolódtak, a rendszer a támadást funkcionális szinten nem érzékelte hibának. A fő különbség az volt, hogy a támadás és a normál forgalom kívülről nehezebben volt megfigyelhető, ami a támadó számára bizonyos szempontból előnyös, a védekező oldal számára pedig új kihívást jelent.

7.6.3 Konfiguráció C: mTLS + ACL – a támadási felület jelentős szűkítése

A 6.3 és 7.4 alfejezetben bemutatott Konfiguráció C a fenti problémákra kínál koncepcionális választ:

- a kliensoldali tanúsítvány (mTLS) révén a broker **egyértelműen azonosítani tudja** a szenzorokat és a többi klienst,
- az ACL-ek segítségével **topic-szinten szabályozható**, hogy egy adott kliens mely csatornákra publikálhat és melyekre iratkozhat fel,
- az anonim hozzáférés (allow_anonymous false) letiltásával azonnal kizárhatók a nem regisztrált, ismeretlen kliensek.

A 7.4-ben elemzett várható hatás összefoglalva:

- a 7.2–7.3-ban bemutatott jogosulatlan publish kísérletek **vagy a TLS handshake során**, vagy az ACL-ellenőrzésnél **elutasításra kerülnének**,
- a legitim szenzorok által küldött adatok zavartalanul áthaladnának, a támadó üzenetei viszont nem kerülnének be a measurements.csv / measurements_tls.csv állományba,
- a támadások így elsősorban a broker naplóiban jelennének meg (sikertelen kapcsolódási/publish kísérletek), nem pedig a feldolgozott mérési statisztikákban.

A Konfiguráció C ezzel **jelentősen szűkíti** a támadási felületet, és a Zero Trust elvek gyakorlati megvalósítása felé mutat: a broker nem feltételez semmiféle implicit bizalmat, minden kliensnek bizonyítania kell a kilétét, és a hozzáférés kizárólag explicit policy-k alapján történik.

7.6.4 Kapcsolat a kutatási kérdésekkel és további irányok

A dolgozat bevezetésében megfogalmazott fő kutatási kérdések közül több is közvetlenül kapcsolódik a 7. fejezet eredményeihez:

- **Milyen támadási felületet jelent egy egyszerű MQTT-alapú IoT-rendszer titkosítatlan, illetve TLS-es konfigurációban?**

A mérések és scenáriók egyértelműen rámutatnak arra, hogy:

- titkosítatlan MQTT esetén a jogosulatlan publish triviálisan megvalósítható,
- TLS mellett a támadás továbbra is kivitelezhető, ha nincs kliensidentitás és ACL; a támadás „csak” titkosított formában zajlik.

- **Milyen védelmi mechanizmusokkal csökkenthető érdemben a támadási felület?**

A TLS bevezetése szükséges, de önmagában nem elégséges. A koncepcionálisan vizsgált mTLS + ACL kombináció tűnik olyan iránynak, amely:

- egyszerre védi a csatornát (TLS) és azonosítja a klienst (mTLS),
- topic-szinten kontrollálja a jogosultságokat (ACL),
- ezzel a gyakorlatban megakadályozza a 7.2–7.3-ban bemutatott típusú támadásokat.

- **Hogyan hatnak a védelmi mechanizmusok a rendszer működésére és teljesítményére?**

A 5. fejezetben bemutatott mérések alapján a TLS 1.3 bevezetése **nem okozott drasztikus teljesítményromlást** az alacsony mintavételezési rátával dolgozó, két szenzoros tesztkörnyezetben. A mTLS + ACL várható teljesítményhatásának részletes vizsgálata azonban további kutatást igényel, különösen nagyobb eszközszám és valós, heterogén hálózati környezet esetén.

A dolgozat gyakorlati részének korlátai között szerepel, hogy a Konfiguráció C implementációja és tesztelése már nem valósult meg teljes egészében; ehelyett a hangsúly a Konfiguráció A és B közötti különbségek empirikus vizsgálatán, valamint a mTLS + ACL megoldás koncepcionális bemutatásán volt. Ebből adódóan a jövőbeli munka lehetséges irányai:

- a mTLS + ACL konfiguráció gyakorlati megvalósítása és mérése nagyobb szenzorparkkal,
- anomáliadetektáló modulok (pl. statisztikai vagy gépi tanulós módszerek) integrálása a collector/analyzer rétegbe,
- a támadási scénáriók bővítése (pl. DoS-szerű túlterhelés, replay támadások, topic-hierarchián alapuló visszaélés).

Összességében a 7. fejezet eredményei megerősítik a dolgozat kiinduló hipotézisét: egy IoT-rendszer biztonságát nem lehet pusztán a csatornaszintű titkosításra (TLS) bízni. A támadási felület érdemi szűkítéséhez **integrált megközelítésre** van szükség, amely a protokollszintű védelmet (TLS, mTLS) és az alkalmazásszintű kontrollt (ACL, naplózás, anomáliadetektálás) egyaránt figyelembe veszi.

8. Összegzés és kitekintés

8.1 Eredmények összefoglalása

A dolgozat célja az volt, hogy feltárja az IoT-rendszerek legjellemzőbb támadási felületeit, és

megvizsgálja, hogy a modern hálózati biztonsági mechanizmusok – különösen a TLS 1.3-ra épülő megoldások – milyen mértékben képesek ezeket mérsékelni. A 2–3. fejezet átfogó képet adott az IoT technológiai háttéréről, az erőforrás-korlátozott eszközök sajátosságairól, valamint az IoT-re jellemző sebezhetőségekről és támadási vektorokról. A szakirodalom alapján rendszereztem az eszköz-, hálózati-, alkalmazási- és backend-/felhőszintű támadási felületeket, és bemutattam több ismert, valós incidens (pl. Mirai-botnet) tanulságait.

A 4. fejezetben részletesen áttekintettem az IoT-ben leggyakrabban használt kommunikációs protokollokat, különös tekintettel az MQTT-re és a CoAP-ra, valamint a hozzájuk kapcsolódó biztonsági rétegekre (TLS 1.3, DTLS, OSCORE). Külön hangsúlyt kapott a Zero Trust modell IoT-környezetre adaptált értelmezése: az eszközüdentitás, a mikroszegmentáció, a minimális jogosultság elve és a folyamatos monitorozás olyan pilléreként jelentek meg, amelyekre a későbbi gyakorlati architektúra-javaslat épül.

Az 5–7. fejezetekben bemutatott szimulációs környezetben két Wokwi-ban futó ESP32-alapú szenzornode MQTT-n keresztül kommunikál egy Mosquitto brokerrel, a forgalmat pedig Python-alapú backend komponensek (collector, analyzer, támadó kliens) kezelik. A baseline és TLS-es mérések eredményei azt mutatták, hogy a TLS 1.3-ra épülő MQTT-kommunikáció a vizsgált környezetben csak mérsékelt többletterhelést jelent: az üzenetküldési ráták a névleges 5 másodperces periódushoz képest elfogadható tartományban maradtak, miközben a kommunikáció bizalmassága és integritása jelentősen javult.

A támadási scénáriók (jogosulatlan publish titkosítás nélkül, TLS mellett, illetve koncepcionálisan mTLS + ACL eseten) rávilágítottak arra, hogy a TLS önmagában nem elegendő a jogosulatlan hozzáférés megakadályozására. A titkosítatlan konfigurációban a támadó kliens gyakorlatilag korlátlanul képes volt legitim szenzor nevében hamis adatokat küldeni, ami a statisztikák torzulásához vezetett. TLS mellett ugyan ez a forgalom már nem volt lehallgatható, de a jogosultságkezelés hiánya miatt a támadás továbbra is sikeres volt. A dolgozatban felvázolt mTLS + ACL alapú megoldás ugyanakkor jól mutatja, hogy a kliensoldali tanúsítványok és a topic-szintű hozzáférés-szabályozás kombinációja képes érdemben szűkíteni a támadási felületet.

A gyakorlati rész egy további fontos eredménye volt egy egyszerű, MI-alapú anomáliadetektálási kísérlet megvalósítása. Az Isolation Forest algoritmussal végzett vizsgálat a normál és támadó MQTT-forgalom elkülönítésére szolgált, és a kontrollált tesztkörnyezetben 98%-os összesített pontosságot, valamint a támadó minták esetében 1,00 recall értéket eredményezett. Ez arra utal, hogy a klasszikus védelmi mechanizmusok mellett egy könnyűsúlyú, gépi tanulás alapú megfigyelő komponens is érdemben hozzájárulhat a korai támadásfelismeréshez.

8.2 A kutatási kérdések és hipotézisek kiértékelése

Az 1.4 fejezetben megfogalmazott fő kutatási kérdés arra irányult, hogy milyen sebezhetőségek jellemzik leginkább az IoT-eszközöket, és ezek ellen milyen hálózati szintű védelmi mechanizmusok és architektúrák nyújthatnak hatékony védelmet. Az elvégzett munka alapján a hipotézisek a következőképpen értékelhetők:

H1 – Az IoT rendszerek támadási felületei elsősorban a nem megfelelően implementált hitelesítési mechanizmusokból, a gyenge alapértelmezett konfigurációkból és az elavult kriptográfiai módszerek használatából erednek. A szakirodalmi áttekintés és a szimulált támadási scenáriók egyaránt megerősítették ezt a feltevést. A titkosítatlan, anonim MQTT-környezetben a gyenge vagy hiányzó hitelesítés ténylegesen lehetővé tette, hogy a támadó kliens legitim eszköznek álcázza magát, és hamis adatokat juttasson a rendszerbe. H1 tehát **alapvetően igazoltnak tekinthető**.

H2 – A modern biztonságos protokollok (pl. TLS 1.3) megfelelő optimalizációval jelentősen csökkentik az IoT-eszközök hálózati szintű sebezhetőségét. A TLS 1.3-ra épülő MQTT-kommunikáció mérési eredményei azt mutatták, hogy a titkosítatlan baseline-hoz képest a biztonsági szint jelentősen nőtt, miközben az üzenetküldési ráta csak kismértékben változott. Ugyan a dolgozatban elsősorban az MQTT + TLS páros gyakorlati vizsgálata történt meg, a szakirodalom alapján DTLS és OSCORE esetén is hasonló trendek várhatók. H2 így **a vizsgált esettanulmány alapján részben, kvalitatív módon igazolható**.

H3 – A Zero Trust modellre épülő, többretegű védelmi mechanizmusok kombinációja hatékony védelmet nyújt az IoT-rendszerek számára. A dolgozatban kidolgozott architektúra-javaslat (mTLS, ACL-ek, topic-szintű policy-k, monitorozás) jól illusztrálja, hogy a Zero Trust elvek hogyan ültethetők át egy MQTT-alapú IoT-környezetre. A mTLS + ACL alapú konfiguráció ugyan elsősorban koncepcionális szinten került bemutatásra, de világosan látszik, hogy a jogosulatlan publish típusú támadások ellen hatékony védelmet nyújtana. H3 ezért **erős elméleti és részben gyakorlati alátámasztást kapott**.

H4 – Az automatizált sebezhetőség-felismerés és MI-alapú anomáliadetekció integrációja javítja a biztonsági incidensek korai felismerését.

A hipotézis vizsgálatához a dolgozat gyakorlati részében egy Isolation Forest alapú anomáliadetektáló modell került kialakításra, amely a normál és támadó MQTT-forgalomból képzett jellemzők alapján jelölte az anomáliagyánús mintákat. A laboratóriumi

tesztkörnyezetben a modell a támadó minták 100%-át felismerte (recall = 1,00), miközben az összesített pontosság 98%, a precision 0,97, az F1-mutató pedig 0,98 volt. Az eredmények alapján H4 a vizsgált szimulációs környezetben igazoltnak tekinthető: az MI-alapú anomáliadetektálás mérhetően javította a támadó üzenetek felismerését, és hatékony kiegészítő védelmi réteget jelentett a hagyományos kommunikációs és hozzáférés-vezérlési mechanizmusok mellett.

8.3 További kutatási irányok

A dolgozatban bemutatott szimulációs környezet és architektúra jó alapot szolgáltat további, mélyebb vizsgálatokhoz. A legfontosabb potenciális folytatási irányok a következők:

- **mTLS + ACL teljes gyakorlati implementációja és mérése:** valós (vagy fizikai) eszközökkel kiegészített tesztkörnyezetben érdemes lenne számszerűsíteni, hogy a kliensoldali tanúsítványkezelés és az ACL-ek milyen overheadet jelentenek, illetve mennyire hatékonyan blokkolják a jogosulatlan hozzáférési kísérleteket.
- **CoAP/DTLS/OSCORE gyakorlati vizsgálata:** a dolgozat elméleti szinten bemutatta ezeket a megoldásokat, de a későbbi kutatásban érdemes lenne hasonló Wokwi- vagy fizikai tesztkörnyezetben mérni a teljesítmény- és biztonsági jellemzőiket, és összevetni őket az MQTT + TLS modell eredményeivel.
- **Valós eszközparkra történő átültetés:** a Wokwi-szimulációt Raspberry Pi, fizikai ESP32 boardok, valamint edge gateway-k bevonásával lehetne tovább vinni, hogy a mérési eredmények közelebb kerüljenek egy ipari vagy okosotthon-szenárióhoz.
- **Fejlettebb anomáliadetektáló modulok vizsgálata és integrálása:** a collector/analyzer rétegbe, különös tekintettel az online, valós idejű detektálásra, a gazdagabb feature-készletre és az összetettebb gépi tanulási modellekre.

Összességében a dolgozat azt mutatta meg, hogy már egy viszonylag egyszerű, szimulált IoT-környezetben is jól vizsgálhatók az alapvető támadási felületek és védelmi mechanizmusok. A TLS 1.3-alapú védelem bevezetése a vizsgált esetben elfogadható teljesítmény mellett nyújt jelentős biztonsági előnyt, ugyanakkor a hosszú távon ellenálló IoT-rendszerekhez elengedhetetlen a Zero Trust szemléletű, többrétegű megközelítés és az automatizált, intelligens védelem továbbfejlesztése.

9. Felhasznált irodalomjegyzék

ANTONAKAKIS, M. et al., 2017. Understanding the Mirai Botnet. In: 26th USENIX Security Symposium (USENIX Security 17). Berkeley, CA: USENIX Association, pp. 1093–1110.

BANKS, A., GUPTA, R., 2014. MQTT Version 3.1.1. OASIS Standard. Elérhető: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.pdf> (hozzáférés: 2026.04.06.).

CERT-EU, 2023. Cyber Security Brief 23-06 – May 2023. Elérhető: <https://cert.europa.eu/publications/threat-intelligence/cb23-06/> (hozzáférés: 2026.04.06.).

COSTIN, A., ZADDACH, J., FRANCILLON, A., BALZAROTTI, D., 2014. A Large-Scale Analysis of the Security of Embedded Firmwares. In: 23rd USENIX Security Symposium (USENIX Security 14). San Diego, CA: USENIX Association, pp. 95–110.

ECLIPSE FOUNDATION, é.n. Eclipse Mosquitto Documentation. Elérhető: <https://mosquitto.org/documentation/> (hozzáférés: 2026.04.06.).

ECLIPSE FOUNDATION, é.n. Eclipse Paho MQTT Python Client Documentation. Elérhető: <https://eclipse.dev/paho/files/paho.mqtt.python/html/> (hozzáférés: 2026.04.06.).

ENISA, 2017. Baseline Security Recommendations for IoT in the Context of Critical Information Infrastructures. Elérhető: https://www.enisa.europa.eu/sites/default/files/publications/WP2017%20O-1-1-2%201%20Baseline%20Security%20Recommendations%20for%20IoT%20in%20the%20context%20of%20CII_FINAL.pdf (hozzáférés: 2026.04.06.).

ENISA, 2018. Good Practices for Security of Internet of Things. Elérhető: <https://www.enisa.europa.eu/sites/default/files/publications/WP2018%20O-1-1-1%201%20Good%20practices%20for%20security%20of%20IoT.pdf> (hozzáférés: 2026.04.06.).

ENISA, 2018. IoT Security Standards Gap Analysis. Elérhető: <https://www.enisa.europa.eu/sites/default/files/publications/WP2018%20O.1.3.1%20IoT%20standards.pdf> (hozzáférés: 2026.04.06.).

ENISA, 2020. Guidelines for Securing the Internet of Things. Elérhető: <https://www.enisa.europa.eu/sites/default/files/publications/ENISA%20Report%20-%20Guidelines%20for%20Securing%20the%20Internet%20of%20Things.pdf> (hozzáférés: 2026.04.06.).

ENISA, 2021. ENISA Threat Landscape 2021. Elérhető: <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2021> (hozzáférés: 2026.04.06.).

ENISA, 2023. ENISA Threat Landscape 2023. Elérhető: <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2023> (hozzáférés: 2026.04.06.).

ETSI, 2020. ETSI EN 303 645 V2.1.1 (2020-06): Cyber Security for Consumer Internet of Things. Elérhető: https://www.etsi.org/deliver/etsi_en/303600_303699/303645/02.01.01_60/en_303645v020101p.pdf (hozzáférés: 2026.04.06.).

FAGAN, M., MEHNTA, J., SCOURAS, Z., 2020. Foundational Cybersecurity Activities for IoT Device Manufacturers. NIST Interagency/Internal Report 8259. Elérhető: <https://nvlpubs.nist.gov/nistpubs/ir/2020/NIST.IR.8259.pdf> (hozzáférés: 2026.04.06.).

MICROSOFT, 2021. Evolving Zero Trust. Microsoft Position Paper. Elérhető: <https://cdn-dynmedia-1.microsoft.com/is/content/microsoftcorp/microsoft/final/en-us/microsoft-brand/documents/Evolving-Zero-Trust-Microsoft-Position-Paper.pdf> (hozzáférés: 2026.04.06.).

MODADUGU, N. et al., 2022. The Datagram Transport Layer Security (DTLS) Protocol Version 1.3. RFC 9147. Elérhető: <https://www.rfc-editor.org/info/rfc9147> (hozzáférés: 2026.04.06.).

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST), 2020. Zero Trust Architecture. NIST Special Publication 800-207. Elérhető: <https://doi.org/10.6028/NIST.SP.800-207> (hozzáférés: 2026.04.06.).

NATIONAL VULNERABILITY DATABASE (NVD), 2021. CVE-2021-35394 Detail. Elérhető: <https://nvd.nist.gov/vuln/detail/CVE-2021-35394> (hozzáférés: 2026.04.06.).

NMHH, 2023. Kutatási jelentés. Elérhető: https://nmhh.hu/dokumentum/247787/nmhh_uzleti_kutatas_2023_riport.pdf (hozzáférés: 2026.04.06.).

OASIS, 2019. MQTT Version 5.0. OASIS Standard. Elérhető: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.pdf> (hozzáférés: 2026.04.06.).

OWASP FOUNDATION, é.n. OWASP Internet of Things Project. Elérhető: <https://owasp.org/www-project-internet-of-things/> (hozzáférés: 2026.04.06.).

PYTHON SOFTWARE FOUNDATION, é.n. Python 3.9 Documentation. Elérhető: <https://docs.python.org/3.9/> (hozzáférés: 2026.04.06.).

RAHMAN, R.A., SHAH, B., 2016. Security analysis of IoT protocols: A focus in CoAP. In: 2016 3rd MEC International Conference on Big Data and Smart City (ICBDSC). Muscat, Oman: IEEE. Elérhető: <https://doi.org/10.1109/ICBDSC.2016.7460363> (hozzáférés: 2026.04.06.).

RESCORLA, E., 2018. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446. Elérhető: <https://www.rfc-editor.org/info/rfc8446> (hozzáférés: 2026.04.06.).

RESCORLA, E., MODADUGU, N., 2012. Datagram Transport Layer Security Version 1.2. RFC 6347. Elérhető: <https://www.rfc-editor.org/info/rfc6347> (hozzáférés: 2026.04.06.).

SCIKIT-LEARN DEVELOPERS, é.n. IsolationForest. Elérhető: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html> (hozzáférés: 2026.04.06.).

SELANDER, G., MATTSSON, J., PALOMBINI, F., SEITZ, L., 2019. Object Security for Constrained RESTful Environments (OSCORE). RFC 8613. Elérhető: <https://www.rfc-editor.org/info/rfc8613> (hozzáférés: 2026.04.06.).

SHELBY, Z., HARTKE, K., BORMANN, C., 2014. The Constrained Application Protocol (CoAP). RFC 7252. Elérhető: <https://www.rfc-editor.org/info/rfc7252> (hozzáférés: 2026.04.06.).

SICARI, S., RIZZARDI, A., GRIECO, L.A., COEN-PORISINI, A., 2015. Security, privacy and trust in Internet of Things: The road ahead. Computer Networks, 76, pp. 146–164.

STATISTA RESEARCH DEPARTMENT, 2024. Internet of Things (IoT) connected devices installed base worldwide from 2019 to 2030. Elérhető: <https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/> (hozzáférés: 2026.04.06.).

WEBER, R.H., STUDER, E., 2016. Cybersecurity in the Internet of Things: Legal aspects. Computer Law & Security Review, 32(5), pp. 715–728.

WOKWI, é.n. ESP32 Simulation. Elérhető: <https://docs.wokwi.com/guides/esp32> (hozzáférés: 2026.04.06.).

WOKWI, é.n. Wokwi Documentation. Elérhető: <https://docs.wokwi.com/> (hozzáférés: 2026.04.06.).